Hantro VC9000E

Video Encoder Wrapper Layer

API User Manual

Document Revision 1.20

19 September 2024

This document is compatible with the following product:

Hantro VC9000E software release version 2.4.123

VERISILICON

LEVEL B: CONFIDENTIAL - DISTRIBUTION RESTRICTED

Legal Notices

Preface

This chapter introduces the *Hantro VC9000E Video Encoder Wrapper Layer API User Manual*.

About This Document

This document introduces the application programming interface (API) of the encoder wrapper later (EWL) of Hantro VC9000E multi-format video encoder IP. It describes the data types, enumerations, structures, and functions in the API, which is written in C code. It also provides a programming overview and examples for your reference.

Additional Reading

- Hantro VC9000E Video Encoder Hardware Features
- Hantro VC9000E Video Encoder Memory Buffer and Format Organization
- ITU-T Rec. H.265: High-efficiency video coding
- ITU-T Rec. H.264: Advanced video coding for generic audiovisual services
- ITU-R REC. BT.601: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios
- ITU-R REC. BT.709: Parameter values for the HDTV standards for production and international programme exchange
- ITU-R REC. BT.2020: Parameter values for ultra-high-definition television systems for production and international programme exchange

Table of Contents

1 Overview

This document describes the application programming interface (API) of the encoder wrapper layer (EWL) for Hantro VC9000E video encoder hardware.

The multi-format VC9000E video encoder can be configured as multi-core to support up to 8192x4096 encoding. It features high quality, high throughput, low power consumption, and negligible load on the host CPU.

The functionality of the VC9000E video encoder is exposed to an application through an encoder API on top of all the codec algorithms and the encoder software. The encoder software is built on an abstraction layer called EWL. The EWL includes the operating system abstraction and the hardware abstraction, and encapsulates all the required system-dependent interfaces. This structure eliminates the need of modifying the encoder software when porting the encoder software to a system.

This document describes the data types, enumerations, structures, and functions in the VC9000E EWL API, which is written in ANSI C. It also provides a guide on how to integrate the VC9000E video encoder software to your system by using the EWL API.

2 Software Integration

The EWL interface describes the service control software requirements of the environment. You can access and implement system-specific functionalities through this interface. The software delivery package provides an implementation example on Linux.

2.1 Control Software Configuration

When compiling the configuration software, you must update some global encoder information to match the system specification. This information is provided in the *enccfg.h* file and you may need to update it for proper configuration.

2.2 Register Access

The encoder hardware is configured using a set of memory-mapped I/O registers. You must map these registers to the address space of the control software and provide read and write access to these registers through the `EWLReadReg()` and `EWLWriteReg()` functions.

When hardware configuration is completed, the `EWLEnableHW()` function enables the hardware with a final writing to one specific register. This specific hardware starting point allows flexible EWL implementation, in which the register accesses can be buffered. Once buffered, the cached values are written to the hardware when `EWLEnableHW()` is called and refreshed by reading back from the registers. This refresh is always executed after a hardware status change, which is signaled by a return from `EWLWaitHwRdy()`.

The software can force the hardware to stop by calling `EWLDisableHW()`. This function disables the hardware by writing to a specific register, whose access should not be buffered.

2.3 Power and Clock Control

To access registers and starts encoding, you must power on the encoder core first. During initialization, the software reads the ID of an encoder core and corresponding configuration information from registers. When encoding one frame, the power and clock of the encoder must be active between calling `EWLReserveHw()` and `EWLReleaseHw()`. When the power and clock is enabled, the software can obtain register values using `EWLReadReg()` or `EWLWriteReg()`. If the register values are used outside the power and clock active period, you must read these values out and stores them into a cached memory area.

The *enccfg.h* file provides the configuration of setting up default clock gating, which enables all clock gatings. When a clock gating is enabled, the clock of the corresponding module will be turned off if the module is not used.

2.4 Hardware and Software Synchronization

Software and hardware components need to synchronize their operations. When the software expects the hardware has completed its current job, it calls `EWLWaitHwRdy()`, which must be blocked until the hardware completes processing.

2.4.1 Interrupt and Polling Methods

The hardware has two modes of communication to indicate that it completes processing: interrupt and polling. It is the integrator who decides which mode to use.

- Interrupt mode:

  In this mode, an interrupt service routine (ISR) must be established. Only after the ISR is triggered by an interrupt from the underlying hardware, VCCORE, can the software be unblocked.

  The specific implementation varies by target operating system and hardware platform.

- Polling mode

  In this mode, the EWL implementation reads relevant hardware status registers in an infinite loop to detect whether the hardware is ready, and exits the loop when the hardware completes processing.

To avoid infinite deadlocks, you can use both software timeout, which is system-specific, and hardware timeout, which is enabled or disabled by the interrupt bit, bit 11, in swreg1, the encoder device configuration register.

In the case of encoder deadlock, you need to wait for `EWLWaitHwRdy()` to return `EWL_HW_WAIT_TIMEOUT` before implementing timeout.

2.5 Memory Allocation in Security Mode

The linear memory is allocated through `EWLMallocLinear()`, with the `EWLLinearMem_t.mem_type` field indicating whether it is the CPU or VPU that accesses the memory and attributes of the memory. Among these attributes, the security attribute indicates whether the memory belongs to the META or CORE memory region. For the META region, the REE CPU can access the memory directly. But for the CORE region, the REE CPU cannot access the memory directly. When the security mode is enabled, security flags are set according to memory usage. For information about these flag, and access scenarios of memories, see Section *Memory Usage*.

When implementing your own memory allocator that supports allocating a buffer from a security or normal region, you can use `EWLGetMemAttribute(mem_type)` on the allocator to get the security attribute and then allocate buffers accordingly.

NOTE: The security feature is not available unless the hardware supports it.

2.6 Memory Synchronization

The memory used by VPU cannot be accessed by CPU directly in some applications. Therefore, the following data synchronization between CPU (host) memory and VPU (device) memory are required:

- Input data synchronization from CPU memory to VPU memory before encoding.
- Output data synchronization from VPU memory to CPU memory after encoding.

To enable memory synchronization, you need to set SUPPORT_MEM_SYNC = y in the globaldefs file.

2.6.1 Related Functions

2.6.1.1 Memory Allocation and Release

The host can use the following EWL functions to allocate and release the device memory:

- `EWLMallocLinear()` for allocating device memory
- `EWLFreeLinear()` for releasing device memory

When synchronization is enabled, you can synchronize the host memory and the device memory through the following two functions to allocate and release the host memory:

- `EWLMemSyncAllocHostBuffer()` for allocating host memory
- `EWLMemSyncFreeHostBuffer()` for releasing host memory

2.6.1.2 Memory Data Synchronization

`EWLSyncMemData()` is called to synchronize the host memory and the device memory in each transfer.

NOTE: This function can be overwritten.

2.6.2 Memory Usage

The following table provides the usage of different memories, with six flags of VPU_RD, VPU_WR, CPU_RD, CPU_WR, EXT_RD, and EXR_WR indicating which IP has access to read or write the memory. You can adjust the access flags according to your applications.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_OUT_SCAL	Stores down-scaling output in YUV format.	**VPU_WR**: VPU writes down-scaling output to the buffer. **CPU_RD**: CPU reads the buffer data and writes it to a file.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_IN_SURFACE	Stores input picture data.	**VPU_RD**: VPU reads the buffer as its input. **EXT_WR**: External IP writes input picture data to the buffer.
EWL_MEM_USAGE_TMP_COEFF	Stores compression coefficients. When scan buffer overflows, the encoder uses the coefficient buffer internally to stores the overflowed data. **NOTE**: Synchronization of this buffer is unecessary.	**VPU_WR** and **VPU_RD**: VPU reads and writes compression coefficients from / to the buffer.
EWL_MEM_USAGE_TMP_RECON (for luma data storage)	Stores reconstructed / reference luma frames.	**VPU_WR**: VPU writes reconstructed luma frames. **VPU_RD**: VPU reads the buffer data as reference luma frames. **CPU_RD**: CPU reads the buffer for debugging and internal testing.
EWL_MEM_USAGE_TMP_RECON (for chroma data storage)	Stores reconstructed / reference chroma frames.	**VPU_WR**: VPU writes reconstructed chroma frames. **VPU_RD**: VPU reads the buffer data as reference chroma frames. **CPU_RD**: CPU reads the buffer for debugging and internal testing.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_TMP_AV1FRMCTX	Stores the entropy context for AV1 frame encoding.	**CPU_WR**: CPU writes the initial or updated context. **VPU_WR**: VPU writes the entropy context data of the current frame. **VPU_RD**: VPU reads the entropy context of the previous frame. **CPU_RD**: CPU reads the context generated by VPU from this buffer.
EWL_MEM_USAGE_TMP_VP9FRMCTX	Stores the entropy context for VP9 frame encoding.	**CPU_WR**: CPU writes the initial or updated context into this buffer. **VPU_WR**: VPU writes the entropy context data of the current frame. **VPU_RD**: VPU reads the entropy context of the previous frame. **CPU_RD**: CPU reads the context generated by VPU from this buffer.
EWL_MEM_USAGE_OUT_SIZETABLE	Stores the NAL unit size table.	**VPU_WR**: VPU writes the size of each NAL unit it generates. **CPU_WR**: CPU writes the size of each NAL unit generated by the software, such as SPS NAL unit and SEI NAL unit. **CPU_RD**: CPU reads the NAL unit sizes to process slices.
EWL_MEM_USAGE_TMP_CTBRC	Stores CTB rate control information. **NOTE**: Synchronization of this buffer is unecessary.	**VPU_WR**: VPU writes CTB rate control information of the current frame. **VPU_RD**: VPU reads CTB rate control information of reference frames.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_OUT_CTBBITS	Stores the bit statistics of each CTB.	**VPU_WR**: VPU writes the number of bits in each CTB. **CPU_RD**: CPU reads and analyzes the bit statistics information.
EWL_MEM_USAGE_TMP_RFCTS	Stores the compression table used by reference frame compression (RFC).	**VPU_WR**: VPU writes the compression table of reconstructed frame. **VPU_RD**: VPU reads the compression table of reference frame. **CPU_WR**: CPU generates and writes a fake compression table if errors occur during encoding. **CPU_RD**: CPU reads the buffer for debugging and internal testing.
EWL_MEM_USAGE_OUT_STRM (for look-ahead encoding)	Stores the stream output of the look-ahead pass-1 encoder as intermediate output. **NOTE**: Synchronization of this buffer is unecessary.	**VPU_WR**: VPU writes the stream output of the look-ahead pass-1 encoder. This stream is a temporary output. **CPU_WR**: CPU writes the header streams generated by the software.
EWL_MEM_USAGE_IN_OVERLAY	Stores overlay input picture data.	**EXT_WR**: External IP or CPU writes the overlay layer data as OSD input. **VPU_RD**: VPU reads overlay input picture data and uses it for alpha blending (OSD) with the encoder input picture data.
EWL_MEM_USAGE_IN_TRANSFORM	Stores the input pictures with color formats converted. **NOTE**: Synchronization of this buffer is unecessary.	For test only.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_IN_SURFACE (for look-ahead encoding)	Stores down-sampled input picture data used to be used by the look-ahead pass-1 encoder.	**EXT_WR** External IP writes down-sampled input picture data. **VPU_RD**: VPU reads down-sampled picture data.
EWL_MEM_USAGE_IN_SURFACE_DECTS	Stores the DEC400 compression table when the encoder input is compressed by DEC400.	**EXT_WR**: DEC400 writes the compression table. **VPU_RD**: DEC400 reads the table to obtain compression information.
EWL_MEM_USAGE_IN_OVERLAY_DECTS	Stores the DEC400 compression table when the overlay input is compressed by DEC400.	**EXT_WR**: DEC400 writes the compression table. **VPU_RD**: DEC400 reads the table to obtain compression information.
EWL_MEM_USAGE_OUT_STRM	Stores output stream of the encoder.	**CPU_WR**: CPU writes header streams generated by the software. **VPU_WR**: VPU writes the stream it generates as the encoding result. **CPU_RD**: CPU reads stream data.
EWL_MEM_USAGE_IN_QPMAP	Stores the quantization parameter (QP) map when it is enabled.	**EXT_WR**: CPU or external IPs write QP map. **VPU_RD**: VPU reads the QP map and parses it to set the QP for the corresponding block.
EWL_MEM_USAGE_IN_CUCTLMAP	Stores the CU control map when it is enabled.	**EXT_WR**: CPU or external IPs write CU control map. **VPU_RD**: VPU reads the CU control map and parses it to obtain the CU control information of the corresponding block.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_IN_CUIDXMAP	(Reserved) Stores the bit map which indicates whether the corresponding CU parameter is valid.	**EXT_WR**: CPU writes the bit map. **VPU_RD**: VPU reads the bit map.
EWL_MEM_USAGE_TMP_EXTSRAM	Serves as a cache and stores cached reference data when the external SRAM feature is enabled. **NOTE**: Synchronization of this buffer is unnecessary.	**VPU_WR**: VPU writes reference data. **VPU_RD**: VPU reads the reference data.
EWL_MEM_USAGE_IN_MVINFO	(Reserved) Stores motion vector (MV) parameters for each block. **NOTE**: This buffer is only available for certain VC9000E versions.	**VPU_RD**: VPU reads the MV parameters. **EXT_WR**: External IP writes parameters of 21 candidate MVs.
EWL_MEM_USAGE_IN_LINEBUF	Serves as a loop-back line buffer for low-latency encoding. **NOTE**: Synchronization of this buffer is unnecessary.	**EXT_WR**: External IP writes input picture data according to the handshaking signal defined between VPU and its up-stream IP. **VPU_RD**: VPU reads input picture data.
EWL_MEM_USAGE_TMP_TMVP	Stores information for temporal motion vector prediction (TMVP). **NOTE**: Synchronization of this buffer is unnecessary.	**VPU_WR**: VPU writes the TMVP information of frames it generates. **VPU_RD**: VPU reads the information of reference frame.
EWL_MEM_USAGE_TMP_MCSYNC	Stores sync_word when multiple encoder cores are used.	**CPU_WR**: CPU initializes the buffer by filling it with the value 0. **VPU_WR**: VPU writes the value of sync_word for the current reconstructed frame. **VPU_RD**: VPU reads the values of sync_word for reference frames.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_OUT_STRM (for re-encoding)	Stores output stream data for re-encoding.	**CPU_WR**: CPU writes header data. **VPU_WR**: VPU writes the encoded stream. **CPU_RD**: CPU reads and processes the output stream data.
EWL_MEM_USAGE_IN_QPMAP (for look-ahead encoding)	Stores delta quantization parameter (QP) map for the look-ahead pass-1 encoder when CuTree is enabled.	**CPU_WR**: CPU initializes the buffer by filling it with the value 0. **VPU_WR**: VPU writes the QP map generated by CuTree. **VPU_RD**: VPU reads the QP map and parses it to set the QP for the corresponding block. **CPU_RD**: CPU reads and clears segcount in the buffer.
EWL_MEM_USAGE_TMP_COST	Stores CuTree analysis results.	**CPU_WR**: CPU initializes the buffer by filling it with the value 0. **VPU_WR**: CuTree writes analysis result. **VPU_RD**: CuTree reads the analysis result it wrote previously.
EWL_MEM_USAGE_OUT_CUINFO	Stores CU information.	**VPU_WR**: VPU writes the encoding parameters, such as partition, mode, and MV, used by the current frame. **VPU_RD**: CuTree reads the CU information for analysis. **CPU_RD**: CPU reads the buffer to process the CU information when required.

Memory Usage Hint	Description	Access Scenario
EWL_MEM_USAGE_TMP_AV1PRC	Serves as a scratch buffer for AV1 stream generation. **NOTE**: Synchronization of this buffer is unnecessary.	**VPU_WR**: VPU writes the stream before the final stream is generated. **VPU_RD**: VPU reads the pre-carry data to generate the final stream.
EWL_MEM_USAGE_TMP_H264COL	Serves as a collocated buffer for H.264. **NOTE**: Synchronization of this buffer is unnecessary.	**VPU_WR**: VPU output of the current frame. **VPU_RD**: VPU reads the reference frame.
EWL_MEM_USAGE_IN_SURFACE (stab)	Stores the next input picture as input to the stabilization module.	**EXT_WR**: External IP writes input data. **VPU_RD**: The stabilization module of VPU reads the buffer.
EWL_MEM_USAGE_TMP_TILECTX	Stores tile de-block information.	**CPU_WR**: CPU clears the buffer by filling it with the value 0 before VPU starts encoding each frame. **VPU_WR**: VPU writes the de-block information of the current tile. **VPU_RD**: VPU reads the de-block information of the current tile.
EWL_MEM_USAGE_TMP_TILEHEIGHT	Stores tile height information. This buffer is used only for low-latency encoding.	**CPU_WR**: CPU writes the height of each tile. **VPU_RD**: VPU reads the height information of each tile.

2.7 Multi-Instance Encoding

The Hantro video encoder is designed as context-free to process one picture frame. After encoding one frame, the hardware resets its internal state. All contexts used to encode one picture are obtained from registers. Such a design makes the encoder capable of encoding multiple streams in a frame-based time-division way.

In the software, each stream is maintained as one instance. You can create multiple software instances to drive one single hardware block on a time-sharing basis. For multi-instance support, the software needs to provide a mechanism to make each instance access the hardware exclusively. Such a mechanism should be implemented by `EWLReserveHw()` and `EWLReleaseHw()`. These two functions grant and release exclusive hardware access to a given software instance.

In the implementation of `EWLReserveHw()`, the instance queries for hardware access. If the hardware is occupied by other instances, the instance waits. When the hardware is available, this function becomes the owner of the hardware, and the corresponding instance can access the hardware exclusively. After the owner instance finishes the hardware-related operation, `EWLReleaseHw()` is called to release the hardware access.

`EWLReserveHw()` and `EWLReleaseHw()` are called by the control-software function `VCEncStrmEncode()`. The reference Linux implementation is provided. The exclusive access is maintained in the Linux kernel driver to make the multi-instance work in both a multi-thread and multi-process manner.

When VCMD is enabled, the hardware access is under the control of VCMD. VCMD pipelines each encoding task into a queue. Each task uses a VCMD buffer. `EWLReserveHw()` and `EWLReleaseHw()` are not used.

3 Linux Kernel Mode Drivers

The verfication of the VC9000E video encoder is on an FPGA board connected with a Linux host PC through Peripheral Component Interconnect Express (PCIe). Therefore the reference driver that works with the hardware is based on Linux and PCIE.

The Linux kernel mode driver interacts with a software development kit (SDK) through EWL. It implements part of the functions required by EWL. EWL for Linux will call the ioctl provided by the Linux kernel driver.

To verify the VC9000E video encoder, the following two Linux kernel mode drivers are used:

- Memory allocator driver

  A kernel driver, also called memalloc, that manages memory for VC9000E series encoders in the test environment. It divides memory into chunks (a block of memory) and assigns base addresses to these chunks.

- VC9000E series device driver

  A kernel driver that interacts with the VC9000E hardware.

NOTE: In the SDK release package, file `<VCESW base>/linux_reference/ewl/ewl.c` is the implementation of the EWL interface for Linux running with the Hantro VC9000E encoder IP.

3.1 Memory Allocator Driver

The memory allocator driver is a reference memory allocator used for the FPGA verification of VC9000E IPs. It allocates and frees memory chuncks, which are blocks of memory outside the Linux memory management, by managing the physical memory addresses from PCIe Base Address Registers (BAR) or the reserved addresses when booting the Linux kernel. When the

compiling option `PCIE_EN = n`, macros `HLINA_START_ADDRESS` and `HLINA_SIZE` in `Makefile` are used to claim the memory space. You must ensure the memory managed by memalloc is used only by the encoder, instead of other devices or Linux system memory.

The following table lists the compiling options and variables in `Makefile` used for the driver.

Option	Description
PCIE_EN	Whether to initialize the DDR address and size from PCIe BAR, or by configuring macros `HLINA_START_ADDRESS` and `HLINA_SIZE`. `y`: (default) initialize. `n`: not initialize.
EMU	Whether to use emulator. `y`: use. `n`: (default) not use.
SUPPORT_DTB	Whether the driver supports reading hardware resources from DTB. `y`: supports. `n`: (default) does not support.
ARM_CROSS_COMPILE	Whether to use cross compiler. `y`: use. In such cases, you need to specify the cross comipler in `Makefile`. `n`: (default) not use.
HLINA_START	Specifies the base address of the reserved memory, when using reserved memory to boot kernel. The defulat value is 0xB0100000.
HLINA_SIZE	Specifies the memory size in MBs. The default value is 256.

The following table lists the module parameters that can be changed when loading the kernel driver.

Parameter	Data Type	Default Value	Description
mem_dev	charp	"memalloc"	The driver name
ddr_offset	ulong	0	The DDR offset from the DDR base address
alloc_size	uint	HLINA_SIZE	The whole DDR size in the board for video
alloc_base	ulong	HLINA_START_ADDRESS	The DDR base address (when `PCIE_EN=n`)
addr_transl	ulong	HLINA_TRANSL_OFFSET	The address differece in CPU bus from VPU bus
vcmd_size	ulong	HLINA_TRANSL_VCMD_SIZE	VCMD buffer pool size
ddr_size	uint	768	The DDR size which can be allocated by user space

The `memalloc_load.sh` script can be used to install memalloc.ko into the kernel. It loads the driver with defined parameters and at the same time changes the access right of the driver to 666 to make the driver can be used directly.

The memalloc driver is located in directory `<VCESW base>/linux_reference/memalloc`.

Following are the example steps to load the driver:

1. Compile the kernel driver in `software/linux/memalloc` by using command `make`.
2. Load the kernel driver with:
   - `./memalloc_load.sh` to obtain the default maximum size in MBs of the linear memory and base address of linear memory from PCIe BAR.
     You do not need to specify `alloc_base` and `alloc_size`.
   - `./memalloc_load.sh alloc_size=400` to specify the maximum size in MBs of the linear memory.
     In such cases, you need to set `PCIE_EN=n` in `Makefile` or set `PCIE_EN=n` when compiling the kernel driver.
   - `./memalloc_load.sh alloc_base=0x42000000` to specify the base address of the linear memory.
   - `./memalloc_load.sh alloc_base=0x42000000 alloc_size=400` to specify both the base address and the maximum size of the linear memory.
3. Debug the kernel driver.
   - Check the device node located in directory `/tmp/dev` by executing the following command:
     `ls /tmp/dev`
   - Check the device driver listed in `/proc/devices` by executing the following command:
     `cat /proc/devices`
   - Check kernel messages by executing command `dmesg`.
   - Check and print more kernel messages for debugging after enabling the messages in `Makefile`.

3.2 VC9000E Series Device Driver

A subsystem consists of a main core and peripherals. The main core can be a VCE core, a VCLE core, a VCEJ core, or a CuTree core (also called IM). The peripherals may include VCMD, DEC400, AXIFE, MMU, etc. You can configure the main core and peripherals according to your need when customizing the hardware. But for the final hardware, the configuration of the main core and peripherals should be set according to the selections and implementation in your system.

The following example shows the typical structure of a Hantro video system with one VCE subsystem and one CuTree subsystem.

```
VCE System
  +- SubSystem0
  |    +- VCE0 (main core 0)
  |    +- VCMD0
  |    +- MMU0
  |    +- AXIFE0
  |
  +- SubSystem1
  |    +- IM1 (main core 1)
  |    +- VCMD1
  |    +- MMU1
  |    +- AXIFE1
```

Before compiling the UMD dirver, you need to configure the subsystems by specifyng the subsystem base address (host physical address) and offests of the main core and every peripherals compared with the base address.

The VC9000E UMD works in the following exclusive modes:

- VCMD mode
  This mode is used when VCMD is integrated in to the VC9000E system and the user wants to control each IP through VCMD.
- Direct mode
  This mode is used when VCMD is not integrated into the VC9000E system or when user wants to control each IP directly instead of through VCMD.

NOTE: The working mode is selected when loading the KMD driver and cannot be changed in run-time.

The VC9000E driver is located in directory
`<VCESW base>/linux_reference/kernel_module/linux`.

The following table provides the files in the directory.

File Name	Description
vcx_driver.c	The entry point
vcx_cfg.h	The definitions of subsystem configurations for VC9000E driver
vcx_normal_driver.c	The code of a direct mode driver
vcx_normal_cfg.h	The subsystem configurations of a direct mode driver
vcx_vcmd_driver.c	The code of a VCMD mode driver
vcx_abnormal_irq.c	The abnormal IRQ processing routines for a VCMD mode driver
vcx_vcmd_cfg.h	The subsystem configurations of a VCMD mode driver
vcx_vcmd_dbgfs.c	dbgfs for a VCMD mode driver
vcx_axife.c	Driver for peripheral AXIFE
vcx_mmu.c	Driver for peripheral MMU

The following table lists the compiling options and variables in `Makefile` used for the driver.

Option	Description
PCIE_EN	Whether to initialize DDR information from PCIe BAR or by configurations. `y`: (default) initialize. `n`: not initialize.
EMU	Whether to use emulator. `y`: use. `n`: (default) not use.
SUPPORT_DTB	Whether the driver supports reading hardware resources from DTB. `y`: supports. `n`: (default) does not support.
ARM_CROSS_COMPILE	Whether to use cross compiler. `y`: use. In such cases, you need to specify the cross comipler in `Makefile`. `n`: (default) not use.
SUPPORT_MMU	Whether the driver supports MMU. `y`: supports. `n`: (default) does not support.
SUPPORT_AXIFE	Whether the driver supports AXIFE. `y`: supports. `n`: (default) does not support.

Option	Description
SUPPORT_VCMD _ENABLE_IP	Whether the driver supports VCMD. y: supports. n: (default) does not support.
SUPPORT_UFBC	Whether the driver supports UFBC. y: supports. n: (default) does not support.
LOW_LATENCY_ SLICEINFO_SUPPORT	Whether the driver supports low-latency encoding by polling slice information. y: supports. n: (default) does not support.
SUPPORT_WATCHDOG	Whether to enable watchdog. y: (default) enable. n: disable.
SUPPORT_DEBUGFS	Whether to enable debugfs for debugging. y: enable. n: (default) disable.
SUPPORT_PM	Whether the driver supports power management. y: supports. n: (default) does not support.

The following table lists the module parameters that can be changed when loading the kernel driver.

Parameter	Data Type	Default Value	Description
vcmd_supported	uint	0	The driver working mode. 0: VCMD driver. 1: direct driver.
vcmd_isr_polling	ulong	0	The mode to wait for device being aborted. 0: use IRQ mode 1: use polling ISR mode.
vsi_kloglvl	int	LOGLVL_ERROR	The kernel driver log level.
enc_dev_n	charp	"vsi_vcx"	The device name
ddr_offset	ulong	0	The memory offset in DDR space
arbiter_urgent	uint	0	The arbiter priority. 0: normal priority. 1: urgent priority.

Parameter	Data Type	Default Value	Description
arbiter_weight	uint	0x1d	Normal priority indicates the required bandwidth. Urgent priority indicates the urgent level.
arbiter_timewindow	uint	0x1d	Time window. 2^n cycles.
arbiter_bw_overflow	uint	0	Whether the bandwidth can overflow. `0`: cannot overflow. `1`: can overflow.

The `driver_load.sh` script can be used to install the vsi_vcx.ko into kernel. It loads the driver with defined parameters and at the same time changes the access right of the driver as 666 to make the driver can be used directly.

3.3 Notes about the Address Space in Driver

When a master IP accesses a device, it will generate the address according to their own address space definition. The host CPU and Hantro video codec can share the same bus address space, e.g. when the CPU is Arm or RISC-V, and the bus is AXI. They can also have their own bus address space and the two spaces are connected by a bridge, e.g. when Hantro video codec and its local memory are on a PCIe card. The FPGA verification board used by VeriSilicon is the second scenario.

There are three kinds of bus in the x86 FPGA verification environement: CPU (x86) system bus, PCIe bus, and AXI bus in FPGA board. When discussing the address space for host CPU, no differene exists between x86 system bus and the PCIe bus because they use the same address space.

Here is some description of the memory space concepts in the PCIe scenario.

From the host CPU perspective, following addresses exist:

- Host memory virtual address
  Memory in the host system is allocated by `malloc()` or `vmalloc()`, and managed by Linux OS through host MMU.

- Host PCIe memory physical address (HPA)
  Memory on the FPGA board. It is used when the CPU accesses it. Parameter `ddr_offest` is used to define the address offset of the DDR corresponding to the base address of corresponding PCIe BAR. The memory allocation and free is maintained by the device driver "memalloc".
- Host PCIe memory virtual address
  Memory on the FPGA board. In user space, it is maintained by the device driver "memalloc". In kernel space, it is maintained by `ioremap_nocache()`.
- Host PCIe hantro registers physical address
  Registers on the FPGA board. They are set up according to the PCIe BAR.
- Host PCIe hantro registers virtual address
  Registers on the FPGA board. In user space, they are maintained by the device driver "memalloc". In kernel space, they are maintained by `ioremap_nocache()`.

In VeriSilicon FPGA board, on board DDR memory is accessed by VC9000E through local AXI bus. Therefore, all addresses issued from the VC9000E subsystem are from the AXI address space.

From the Hantro codec perspective, following addresses exist:

- Device PCIe memory bus address (DBA)
  Memory on the FPGA board. Such memory is used when master IPs on board, such as VC9000E and VCMD. access it. The memory address is fixed and is defined by parameter `ddr_offest` in KMD. On VeriSilicon FPGA, the parameter value is `0`.
- Device PCIe memory virtual address (DVA)
  Memory on the FPGA board. Such memory is used when VeriSilicon MMU is on the board and master IPs such as VC9000E and VCMD on board access the memory. The memory is maintained by VeriSilicon MMU driver.
- Device PCIe Hantro registers bus address
  Such a memory address is fixed. The memory is used when VCMD IPs on board access the registers in their own subsystem. In such a case, only the offset in LSB is valid.

The conversion logic between HPA and DBA is as follows:

DBA = HPA - TanslationOffset,

where TanslationOffset is defined as a KMD module parameter `addr_tranl`. For a PCIe driver, the parameter is invalid, and it is obtained from the information of the PCIe memory BAR.

When the CPU can access DDR and Hantro video encoder subsytem directly (not through PCI), user can compile KMD with option `PCIE_EN=n` when make. In such an option, the DDR and register addresses will be specified by the configuration file or module parameters.

3.4 Guideline for FPGA Verfication

1. Make sure whether the PCI device is probed correctly.

   When the bit file works correctly and user should see similar message when run PCI utility "lspci" in Linux terminal. ``` $ lspci ... 01:00.0 Non-VGA unclassified device: Xilinx corporation Device 8014 Subsvstem:XilinxCorporatlon Device 7 Flags: bus master,fast devsel,latency 0,IRQ 16 Memory at 80000000(64-bit, prefetchable)[size=512M] Memory at a0000000(64-bit, prefetchable)[size=256M] Memory at b0000000(64-bit, prefetchable)[s1ze=16M] Capabilitles:<access denied> ... ```

   where the number 8014 is the ID for the Hantro FPGA board.

   When you observe the similar information, you can proceed to the next step.

2. Modify `vcx_cfg.h` to configure KMD register offsets in each subsystem.

3. Install kernel mode drivers.

4. Compile user space testbench.

   Compile the SDK for PCIe.

5. Run command lines provided in test cases.

   Following are the command lines for entry-level test. They encode 10 frames with fixed QP. The encoding slice types are increased step by step, from encoding I-frames only to encoding I-/P-/B- frames.

   - I-frame only encoding ``` ./video_testenc -i foreman_cif.yuv -w352 -h288 -R1 -b9 -q26 -o stream.hevc ```
   - P-frame only encoding ``` ./video_testenc -i foreman_cif.yuv -w352 -h288 –gopSize=1 -R30 -b9 -q26 -o stream.hevc ```
   - Hierarchy B frame with GOP structure 4 (when the hardware supports B-frame encoding) ``` ./video_testenc -i foreman_cif.yuv -w352 -h288 –gopSize=4 -R30 -b9 -q26 -o stream.hevc ```

   Besides the above command lines, you can explore more test cases provided by VeriSilicon hardware team and create your own test cases according to the *Hantro VC9000E Video Encoder Testbench User Manual* document.

6. Verify the test result.

   The correctness of VC9000E is verified through the comparison betweem the stream generated by FGPA and the stream generated by Cmodel. The two streams should be bit-exactly the same, if the software used to compile the binary for FPGA verification is the

same as the software source code used to compile the Cmodel except the EWL, and the same command line options are used.

In most cases, modifications that do not impact the register settings will not impact the stream generation. Only when the stream generation is not impacted, the bit-exact comparison method is valid. Therefore, it is recommended to verify whether the modification impacts the stream generated after you modify the source code to generate a testbench.

For information about FPGA verification process, refer to *Hantro VC9000E Series Video Encoder FPGA Verification Guide*.

4 Performance Checking and Issue Debugging

4.1 Status and Debugging Registers

Both the main core and its pheripherals have status registers to report the status of the hardware and the source of the exceptions. This section provides the status information, status registers, and debugging registers for VC9000E.

4.1.1 Status Information

When `EncAsicCheckStatus_V2()` is called, the following status will return.

- `ASIC_STATUS_FRAME_READY`: Indicates the encoding of one frame is completed correctly.
- `ASIC_STATUS_SLICE_READY`: Indicates that the encoding of one slice is completed when multi-slice is enable. The hardware continues encoding the remaining picture rows.
- `ASIC_STATUS_FUSE_ERROR`: (Deprecated) Indicates that the feature not supported by the hardware is set up in the register.
- `ASIC_STATUS_SEGMENT_READY`: Indicates that one segment stream buffer is full.

  NOTE: This feature is only available for VCEJ after it is selected during hardware configuration.
- `ASIC_STATUS_LINE_BUFFER_DONE`: Indicates that one segment of an input picture is read out and the corresponding buffer becomes unused and can be overwritten by new data.

  NOTE: The feature is only available in the input low-latecny mode.

- `ASIC_STATUS_POLL_SLICEINFO_TIMEOUT`: Indicates that the input picture data is not available for a long time.

  NOTE: The feature is available when input low-latency DDR polling hardware is configured and ASIC works in such a mode.

- `ASIC_STATUS_UFBC_DEC_ERR`: Indicates that an error occurs from the external frame buffer compressor (UFBC).

- `ASIC_STATUS_BUFF_FULL`: Indicates that the stream buffer is too small and stream writing has achieved or will achieve the end of the stream buffer. In such a condition, you can drop the frame or reencode the frame.

- `ASIC_STATUS_HW_TIMEOUT`: The frame encoding job is not finished correctly due to the internal hardware watchdog timer has reached its threshold. The timer will be reset periodically when hardware pipeline is working correctly. Timeout happens when the hardware is stalled for an unknown reason for a long time.

- `ASIC_STATUS_ERROR`: Indicates that the AXI bus reports errors.

- `ASIC_STATUS_HW_RESET`: Indicates that the core is reset by writing **swreg5[bit0]** from `1` to `0`.

4.1.2 Status Registers

The table below lists the corresponding status registers in VC9000E.
You can clear a status register by writting `1` to its corresponding bit. Writing `0` to the bit does not change the status register value or the corresponding signal.
NOTE 1: The prefix **HWIF_ENC_** is omitted for each register name.
NOTE 2: The UFBC status is obtained from the status register of an external FBC IP.
NOTE 3: The Type register will control if the interrupt is abnormal or normal interrupt when output to VCMD.

Status Regsiter	Swreg Bit	Type & Enable Register
FRAME_RDY_STATUS	swreg1[2]	IRQ_TYPE_FRAME_RDY Enabled by default.
SLICE_RDY_STATUS	swreg1[8]	IRQ_TYPE_SLICE_RDY Controlled by SLICE_INT (swreg3[3])
STRM_SEGMENT_RDY_INT	swreg1[12]	IRQ_TYPE_STRM_SEGMENT Controlled by STRM_SEGMENT_INT (swreg3[1])

Status Regsiter	Swreg Bit	Type & Enable Register
IRQ_FUSE_ERROR	swreg1[9]	(Depracated) IRQ_TYPE_FUSE_ERROR Enabled by default.
IRQ_LINE_BUFFER	swreg1[7]	IRQ_TYPE_LINE_BUFFER Controlled by LINE_BUFFER_INT
TIMEOUT	swreg1[6]	IRQ_TYPE_TIMEOUT Controlled by TIMEOUT_INT (swreg1[11])
BUFFER_FULL	swreg1[5]	IRQ_TYPE_BUFFER_FULL Enabled by default.
SW_RESET	swreg1[4]	IRQ_TYPE_SW_RESET Enabled by default.
BUS_ERROR_STATUS	swreg1[3]	IRQ_TYPE_BUS_ERROR Enabled by default.
INPUT_SEGMENT_POLL_TIMEOUT	swreg118[0]	IRQ_INPUT_SEGMENT_POLL_TIMEOUT Controlled by INPUT_SEGMENT_POLL_TIMEOUT_INT

4.1.3 Debugging Registers

You can obtain the hardware performance from swreg82. When the encoding of a frame is completed, the value in the register represents the cycles (in the unit of encoder clocks) used to encode the frame. The performance target for VC9000E is 250 cycles per MB (each MB is a 16x16 block). You can calculate the performance using swreg82/mbNum. If the result is close to the target, the performance meets the target as well.

NOTE: Different versions of hardware may have different average performance data. 250 cycles/MB is a general guide line. When picture resolution is too small, the pipeline stages may not be well-utilized, which leads to a bad performance.

Although VC9000E has optmized the design to adapt to greater bus latency, performance will decrease if the latency is above the designed limit. To check the decreased performance due to bus latency, you can use the registers listed in the following table.

NOTE: Initial bus latency tolerance is designed as 1000 cycles for HEVC and 750 cycles for H.264. When bus latency is larger than these values, performance will be impacted. You can consult the hardware team for the current status.

Register	Swreg	Description
AXI_STRM_WRITE_PENDING	swreg308	EMC stream AXI write pending
AXI_RECON_WRITE_PENDING	swreg309	Reconstruced pixel AXI write pending

Register	Swreg	Description
AXI_REC4N_WRITE_PENDING	swreg310	Reconstruced 4n pixel AXI write pending
AXI_PRP_READ_PENDING	swreg311	Pre-processing AXI read pending
AXI_REF_READ_PENDING	swreg312	Reference frame read pending
AXI_REF4N_READ_PENDING	swreg313	Reference frame 4n AXI read pending
AXI_RCROI_READ_PENDING	swreg314	RCROI AXI read pending
AXI_READ_CHANNEL_PENDING	swreg315	AXI read channel pending
AXI_WRITE_CHANNEL_PENDING	swreg316	AXI write channel pending
AXI_TOTAL_PENDING	swreg317	Total AXI bus pending

AXI bus status is useful for checking the working status of an IP. If all transaction are processed correctly, the amount of data requested should be the same as the amount of data trasfered. The following table lists the registers to check whether AXI transactions are executed properly.

Register	Swreg	Description
HWIF_ENC_TOTALARLEN	swreg215	ARLen total, accumulated ARLEN+1
HWIF_ENC_TOTALR	swreg216	R total, RVALID & RREADY
HWIF_ENC_TOTALAR	swreg217	AR total, ARVALID & ARREADY
HWIF_ENC_TOTALRLAST	swreg218	RLast total, RVALID & RREADY & RLAST
HWIF_ENC_TOTALAWLEN	swreg219	AWLen total, accumulated AWLEN+1
HWIF_ENC_TOTALW	swreg220	W total, WVALID & WREADY
HWIF_ENC_TOTALAW	swreg221	AW total, AWVALID & AWREADY
HWIF_ENC_TOTALWLAST	swreg222	WLast total, WVALID & RREADY & WLAST
HWIF_ENC_TOTALB	swreg223	B total, BVALID & BREADY

Bandwidth requirement is also an important factor that affects performance. The registers above can also be used to check the bandwdith cost after encoding a frame. In most cases, swreg215 and swreg219 represent the read bandwidth and write bandwidth, respectively.

4.2 Interrupt Service Routine for Subsystem with VCMD

When VCMD is used in subsystem, the interrupt logic is handled or bypassed by the VCMD module.

The following figure shows the subsystem built by the VCMD and VPU core. **XINT_VCMD** is the interrupt signal to the interrupt controller. The controller then triggers the CPU to process the interrupt.

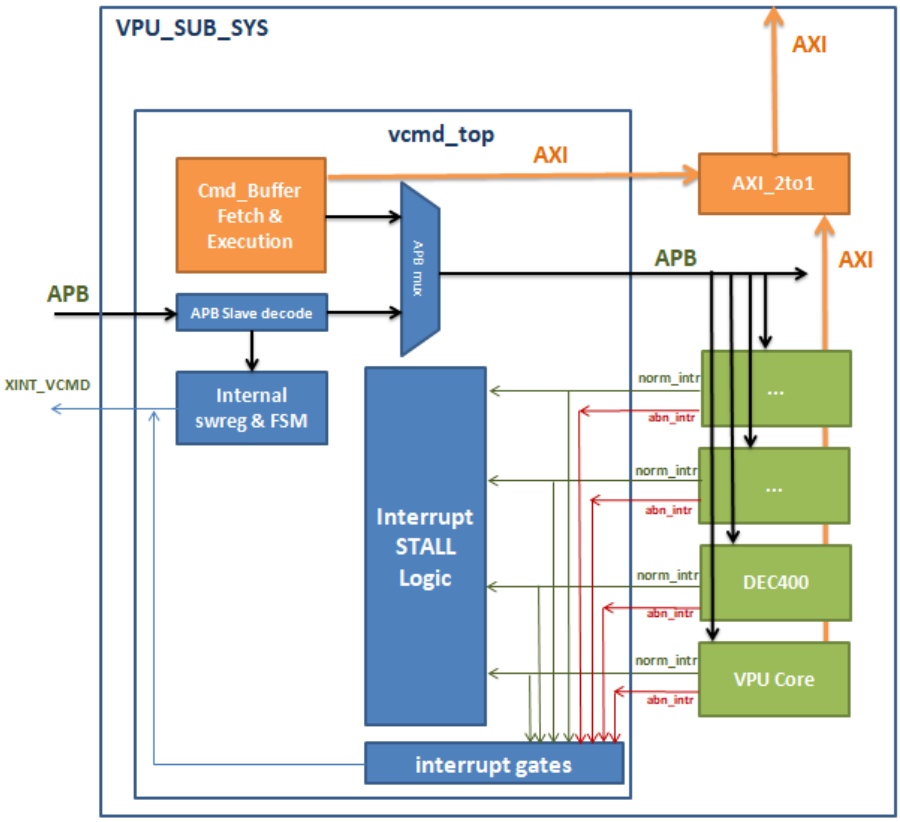

Figure 4.1 Interrupt Signal in the Subsystem with VCMD

As the figure shows, there are two sources of interrupts for signal **XINT_VCMD**. One is the internal interrupts triggered by internal software registers and FSM. These interrupts are generated by some special commands or internal exceptions. The other is the external interrupt. These are the interrupts captured from the modules attached to the VCMD. The VCMD accepts such interrupts, e.g. from the VCE.

The two sources together generate a single interrupt to the CPU.

This document will describe the interrupts source from VCMD and how encoder control software process the interrupts.

In VC9000E, most interrupt source has the corresponding "IRQ Type" register, which controls the normal or abnormal types of interrupts. If the type of the interrupt enters the VCMD is normal, the VCMD will exit the "STALL" command. Otherwise, the VCMD will not exit the "STALL" command and the interrupt will be processed by CPU.

The "STALL" command is defined as an operation code executed by the VCMD. When the command is executed, the VCMD enters the "stall" state and waits to be waken up by an external

normal interrupt. After being waken up, the following commands will be executed. Such a normal interrupt is gated by the VCMD and does not trigger ISR of the CPU. It should be handled by the VCMD according commands in the CMD buffer. Currently, VC9000E only defines the slice-ready interrupt as abnormal when low-latency is required to fetch the ready slice data.

4.2.1 VCMD Interrupts

The VCMD generates its own interrupt. At the same time, the VCMD accepts the bypassed abnormal interrupts from external sources, such as VCE or other peripheral IPs.

The following table lists the interrupt sources generated by the VCMD.

Interrupt Source	Enable Control	Description
vcmd irq_jmp	Yes	Triggered by a JMP command.
vcmd irq_abort	Yes	Triggered by writting 0 to register **sw_start_trigger**.
vcmd irq_cmderr	Yes	Triggered when an illegal command is detected.
vcmd irq_timeout	Yes	Triggered by timeout.
vcmd irq_buserr	Yes	Triggered by error response from the AXI bus.
vcmd irq_endcmd	Yes	Triggered by an END command.
vcmd irq_cmdbuf_timeout	Yes	Triggered by timeout counter when process an command buffer, replace software watchdog (version 1.5.10)
extern abnormal	Yes	An abnormal interrupt from the devices attached to the VCMD

Normal interrupts from the VCE does not generate interrupts to the CPU. VCE normal interrupts, such as frame-ready interrupt, timeout interrupt, and buffer-full interrupt, will wake up the VCMD from the stall state. After the VCMD is waken up, the commands in the CMD buffer can issue interrupts by executing a JMP (controlled by its IE flag) or END command. To reduce the CPU load, the software can disable some normal interrupts and send them after accumulating enough results.

The VCMD interrupts use register **sw_ext_abn_intr_src** and **sw_ext_norm_intr_src** to indicate which interrupt is detected from external modules. When an abnormal interrupt is triggered, the CPU can check the registers to determine the source of the external interrupt and then check the status of the external module to obtain the reason for the triggering of the interrupt.

The external interrupt can be disabled by VCMD register **sw_ext_abn_intr_gate** and **sw_ext_norm_intr_gate**.

Note: In VCMD version 1.5.10, subsystem reset is designed as an error recover mechnism. Note: In VCMD version 1.5.10, AXI2To1 is AXI interface of the subsystem. It is another External abnormal interrupt source. When this external abnormal interrupt happens, kernel driver will reset the whole sub-system by controlling subsystem reset registers. The software will do the context recovery internally.

4.2.2 Interrupt Processing

User space software prepares the CMD buffer that contains commands and data for the encoding of a frame. The `HANTRO_IOCH_LINK_RUN_CMDBUF` ioctl issues the kernel one ready CMD buffer. The buffer is then run directly or linked to a list to run after the previous tasks are completed. The `HANTRO_IOCH_WAIT_CMDBUF` ioctl is called to wait for the execution of the CMD buffer. In most cases, the last command in a CMD buffer is JMP. It will jump to the next CMD buffer or wait for the next CMD buffer to be linked.

The VCMD triggers an interrupt according to its own status or the external status. Such status is processed in the ISR. When the wake-up API in ISR is called, the waiting thread is waken up to check whether the execution of its CMD buffer is completed. This section describes the ISR and related kernel operations and introduces how user space code processes the received status filled by the kernel driver.

In the interrupt service routine, the VCMD status register is checked for the casue of the interrupt.

- For previous CMD buffers They are marked as completed with the "OK" flag, and the owner of the corresponding CMD buffer is waken up if it sleeps in the waiting queue.
- For the current CMD buffer It is marked as completed with a flag indicating the cause.
  A special case is the timeout interrupt. It starts a timer of 300-400 ms to wait for the VCMD abortion interrupt.
  - If the abortion interrupt is not triggered, external timeout will be reported to the system driver to reset the VCMD subsystem.
  - If the abortion interrupt is triggered, the driver will restart the VCMD from this CMD buffer.

The wakeup event is checked in the kernel in each context which entered by calling ioctl `HANTRO_IOCH_WAIT_CMDBUF`. If the event is the one it is waiting for, the user space API which issues the ioctl will be waken up. Then, the corresponding process will start in the user space.

In user space, `VCEncStrmEncodeExt()` processes the encoding of a frame as follows:

1. The API prepares the CMD buffer for the encoding.
2. The CMD buffer is linked to the VCMD work list or is run directly in the kernel.
3. The user space waits for the event from the kernel by calling `VCEncStrmWaitReady()`.
   Such a waiting is waken up when the CMD buffer is maked as "done" with the status flag and wakes up the corresponding waiting-queue.
4. The user space code processes the result of VCMD and VCE according to the status of the CMD buffer and VCE.
5. When a result is returned from `VCEncStrmWaitReady()`, furthur processing, such as rate control adjustment, will be completed in `VCEncStrmEncodeExt()`.
   You can obtain the encoding result and status from the return code and output parameters.

4.3 Debugging Tips

4.3.1 Driver Loading

To check whether the driver is loaded properly, you can use the following methods:

- Use "lspci" to check whether the FPGA board is probed corrctly.
  - Check whether the correct bit file is loaded onto FPGA.
  - Check whether the connection between the FPGA board and power supply is established.
- Use "dmsg" to check whether "memalloc" and "vsi_vcx" drivers are loaded successfully.
  - When the register address space is reserved by other device driver, the loading of the vsi_vcx driver fails.
    Such a case may occur if other driver reserved to region or the driver is removed incorrectly in the last test.
  - When memory un-loading fails, the Linux needs to be rebooted in most cases.

4.3.2 Testbench Command Line Running

When an error occurs and the stream is not generated correctly, some general phenomena is listed and corresponding flow to process them are suggested.

- Testbench prompts some checking issues for the command line The hardware configuration does not support the specified feature, for example,
  - The input formats are not supported.
  - B-frame is not supported.
  - 10-bit encoding is not supported.

For details about the supported features, refer to *Hantro VC9000E Video Encoder TestBench User Manual*.

- Core dumps or memory access fails
  - Hardware initialization fails.
    If AXIFE is selected, AXIFE needs to be configured when compiling the UMD and KMD driver.
  - The software fails to catch the setting conflicts in command lines or it has a bug.
    For such cases, report them to VeriSilicon through JIRA.
- Hardware reports exceptions
  - Dump the registers log and parse it with the utitlity in `software/showregval`.
    For how to compile it, see `readme.txt`.
  - Check the status and do corresponding debugging according to Section *Status and Debugging Registers*.
- Report the issue to VeriSilicon through JIRA with related resources attached.

When report issues, following resources are helpful for the reproduction and quick processing of issues:

- The software package name to verify the IP.
- The command line that reveals the issue.
- Related resource files, such as input picture file, ROI map file, OSD file.

- The log file when the issue happens.

The log file is helpful because it contains the following information:

- General stdout output
- Backtrace
- Register dump result, which is important to hardware debugging (add `-dumpRegister=1 -logOutLevel=6 -logTraceMap=2`).
- More information can be output by increasing the log level and setting more logging trace map bits.
  You can enable all outputs with the option `-logOutLevel=6 -logTraceMap=127`.

When checking register values, we will meet three types of register:

- Address registers
  The "BASE" or "MSB" in the name of such registers indicates the start address of the register is LSB or MSB, respectively. Compared with register values dumped, the address registers may be quite different. Generally the values should not be zero if Cmodel register value is not zero.
- Read-only registers
  Such registers are ID or fuse registers. They should have the same value and the value is not changed among frames.
- Control registers
  Such registers can be read and write. Their values should be the same when encoding the same frame.

Control registers are more important for debugging. You need to check which field has a different value and checking whether it relates to a feature. Register dumpping can happen before encoding and after encoding. Some registers are written by the encoder. Therefore it is necessary to compare the register values both before encoding and after encoding.

4.3.3 Testbench-Encoded Stream Checking

When using the same command line, the Cmodel binary should generate the same stream file as the testbench binary compiled for FPGA verification. For requirements on such a checking, see kmd_ug.

Following are the methods to check whether the stream generated by the Cmodel is correct.

- Use stardard decoder to check whether the stream conforms to the starndard.
  - HEVC: HM
  - H264: JM
  - VP9: VPXDEC
  - AV1: AOMDEC
  - JPEG: libjpeg6b
- Use FFmpeg to decode the stream.
  This method is used for JPEG lossless processing, which is not available for standard JPEG decoder.
- Use professional tools, such as VQ Analyzer provided by ViCueSoft (http://www.vicuesoft.com).
  The tool provides the details of parameter sets, slice header information, reference relationship, and results of each decoding stage, such as prediction mode and deblocking filter.
  If syntax has errors, the tool will fail to parse the stream. Sometimes, the tool can show the locations of errors.

If an error stream is generated, report it to VeriSilicon and provide the materials mentioned in *Testbench Command Line Running*.

4.3.4 Performance Checking

The encoder performance is the combination of hardware and software performance. For hardware performance, you can check register **swreg82**.

The following elements affect the hardware performance.

- Bus latency
  You can check registers swreg308 to swreg317 to see whether the register values are high.
- Encoder entropy engine
  If too many coefficients are generated, the entropy engine (CABAC) will hit the bottleneck. Such a case often happens when the QP is very small, for example, smaller than 16. Therefore, the stream size becomes huge and the CABAC engine cannot process it quickly. The average performance of CABAC is 1 bit/cycle. For example, when the encoder runs at 500MHz and encodes with 30fps, its performance will decrease if the size of one frame is larger than (500MHz/8 bits/30fps) MBs.
- Bandwidth
  You can check the bandwidth using status and whether the system provides such a bandwidth through the values of swreg215 and swreg219.
  In addition, you can try to enable and disable the reference frame compression (RFC) option to see whether the performance changes. If you are using testbench, you can enable RFC with `-compressor=3` or disable it with `-compressor=0`.

The software performance depends on the performance of `VCEncStrmEncode()`. A log in testbench shows the total microseconds used to encode a frame. The following log show the total time for this function (including both the hardware and software).

```
=== Encoding 0 foreman_cif_short.yuv ...
=== Encoded 0 bits=99208 totalBits=99896 averageBitrate=2996880 HWCycles=125658 codingType=0
=== Time(us 11488 HW+SW) ===
```

Where "HWCycles=" indicates the value in **swreg82**, and "Time(us 11488 HW+SW)" shows the total time to encode the frame.

To get the software running time, you can minus the hardware time (swreg82/VPU_MHz).

In one-pass encoding, the average CPU cycles to encode one frame is about 200K to 300K cycles.

In our kernel mode driver, debugfs can be used to check register values and performance statistics. To include debugfs, user should add "SUPPORT_DEBUGFS=y" when compile kernel mode driver. After install vsi_vcx driver, two debugfs file nodes ("regprint" and "performance") will be generated. You can use the cat command to check related information then.

- View register information of each VCMD sub-system in real time:
  - cat /sys/kernel/debug/vsi_vcx/regprint
- View performance statistic of each VCMD sub-system:
  - cat /sys/kernel/debug/vsi_vcx/performance

In the performance file, you will see the utilization of each main core under every VCMD.

4.4 DEC400 Debug Tips

4.4.1 Common issues with DEC400

The mismatch in the configuration of compression parameters between the compression and decompression ends leads to inconsistency between the extracted YUV and the original image. The compression parameters mainly include the following:

- Compression Format
- Tile Mode
- Tile Alignment
- Bit Depth

The first three compression parameters on the decompression side are set through the register gcregAHBDECReadConfig. Bit depth is set through gcregAHBDECReadExConfiguration. If the above issues occur, please first check whether the values of gcregAHBDECWriteConfig and gcregAHBDECWriteExConfig on the compression side are consistent with those of gcregAHBDECReadConfig and gcregAHBDECReadExConfig. If there is inconsistency, please adjust the register configuration on the decompression end to ensure consistency.

DEC400 is compressed and decompressed in units of 256 or 128 bytes (tileSize). Therefore, the YUV input during dec400 compression needs to be aligned according to the requirements of the tilemode. Tilemode is divided into rasters and tiles Assuming using a tilemode of 256x1r, the width of the input YUV needs to be aligned to 256 bytes. Assuming a 32x4 tilemode is used, that is, the width of the input YUV needs to be aligned to 32 bytes and the height needs to be aligned to 4 bytes.

4.4.2 Registers Overview

The registers involved in the code are mainly the following:

- gcregAHBDECControl: mainly uses bit0 bit. When bit0 is set from 0 to 1, software will start flushing all streams.
  - GcregAHBDECIntrEnable: Enable interrupts
  - GcregAHBDECIntrenableEx2: Enable interrupts

  - GcregAHBDECIntrAcknowledgeEx2: Mainly uses bit0 bit. When bit0 changes from 0 to 1, all streams are refreshed and the bit0 bit is also cleared.

- Read Client Configuration
  - GcregAHBDECReadConfiguration: Configure Compression Format, Tile Mode, and Tile Alignment.
  - GcregAHBDECReadExConfiguration: Configure Bit Depth.
  - GcregAHBDECReadBufferBase: Configure the base address of the input buffer
  - GcregAHBDECReadBufferBaseEx: Configure the high bits of the input buffer's base address
  - GcregAHBDECReadBufferEnd: Configure the end address of the input buffer
  - GcregAHBDECReadBufferEndEx: Configure the high bits of the end address of the input buffer
  - GcregAHBDECReadCacheBase: Configure the base address of the tile status buffer
  - GcregAHBDECReadCacheBaseEx: Configure the high bits of the base address for the tile status buffer

The document for Dec400 mentions that the gcregAHBDECIntrKnowledge bit [31] is an AXI BUS ERROR interrupt. But I tried many methods before, including setting the buffer address to all zeros, but none of them triggered the AXI BUS ERROR interrupt. Therefore, the code did not handle this error interrupt.

5 Module Documentation

5.1 Core Information

Marco Definition

- #define ASIC_SWREG_AMOUNT 512

  The total register number of an encoder core.
- #define HW_ID_PRODUCT_MASK 0xFFFF0000

  The bit mask to obtain the product ID from the hardware ID.
- #define HW_ID_MAJOR_NUMBER_MASK 0x0000FF00

  The bit mask to obtain the major version number from the hardware ID.
- #define HW_ID_MINOR_NUMBER_MASK 0x000000FF

  The bit mask to obtain the minor version number from the hardware ID.
- #define HW_ID_PRODUCT(x) (((x & HW_ID_PRODUCT_MASK) >> 16))

  The product ID derived from the hardware ID.
- #define HW_ID_MAJOR_NUMBER(x) (((x & HW_ID_MAJOR_NUMBER_MASK) >> 8))

  The major version number derived from the hardware ID.
- #define HW_ID_MINOR_NUMBER(x) ((x & HW_ID_MINOR_NUMBER_MASK))

  The minor version number derived from the hardware ID.
- #define HW_ID_PRODUCT_H2 0x4832

  The hardware engine is H2, which is the first generation video encoder.
- #define HW_ID_PRODUCT_VC8000E 0x8000

  The hardware engine is VC8000E.
- #define HW_ID_PRODUCT_VC9000E 0x9000

  The hardware engine is VC9000E.
- #define HW_ID_PRODUCT_VC9000LE 0x9010

  The hardware engine is VC9000LE.
- #define HW_ID_PRODUCT_VC9800E 0x9800

  The hardware engine is VC9800E.

- #define HW_PRODUCT_H2(x) (HW_ID_PRODUCT_H2 == HW_ID_PRODUCT(x))

  Specifies whether the product ID is H2 series.
- #define HW_PRODUCT_VC8000E(x) (HW_ID_PRODUCT_VC8000E == HW_ID_PRODUCT(x))

  Specifies whether the product ID is VC8000E.
- #define HW_PRODUCT_SYSTEM60(x)

  Specifies whether the product ID is VC8000E version 6.0. For details, see Section Macro Definition.
- #define HW_PRODUCT_SYSTEM6010(x) (HW_PRODUCT_SYSTEM60(x) && (HW_ID_MINOR_NUMBER(x) == 0x10))

  Specifies whether the product ID is VC8000E version 6.1.
- #define HW_PRODUCT_VC9000(x) (HW_ID_PRODUCT_VC9000E == HW_ID_PRODUCT(x) || HW_ID_PRODUCT_VC9800E == HW_ID_PRODUCT(x))

  Specifies whether the product ID is VC9000E.
- #define HW_PRODUCT_VC9000LE(x) (HW_ID_PRODUCT_VC9000LE == HW_ID_PRODUCT(x))

  Specifies whether the product ID is VC8000LE.
- #define MIN_ASIC_ID_WITH_BUILD_ID 0x80009100

  The ASIC_ID, which indicates when the hardware can support BUILD_ID.
- #define HWIF_REG_BUILD_ID (509)

  The register offset of the build ID register.
- #define HWIF_REG_BUILD_REV (510)

  The register offset of the build revision register.
- #define HWIF_REG_BUILD_DATE (511)

  The register offset of the build date register.
- #define HWIF_REG_CFG1 (80)

  The hardware register configuration_1.
- #define HWIF_REG_CFG2 (214)

  The hardware register configuration_2.
- #define HWIF_REG_CFG3 (226)

  The hardware register configuration_3.
- #define HWIF_REG_CFG4 (287)

  The hardware register configuration_4.
- #define HWIF_REG_CFGAXI (319)

  The hardware register configuration_AXI.
- #define HWIF_REG_CFG5 (430)

  The hardware register configuration_5.
- #define HWIF_CFG_NUM 7

  The number of hardware configuration registers.

- #define CORE_INFO_MODE_OFFSET 31

  The bit position that specifies core mode in the core information words.

- #define CORE_INFO_AMOUNT_OFFSET 28

  The start bit position that specifies the core amount in the core information words.

- #define MAX_SUPPORT_CORE_NUM 4

  The maximum number of encoder cores that the software can support.

- #define ASIC_STATUS_SEGMENT_READY 0x1000

  The bit mask for the stream segment ready flag.

- #define ASIC_STATUS_FUSE_ERROR 0x200

  The bit mask for the fuse error flag.

- #define ASIC_STATUS_SLICE_READY 0x100

  The bit mask for the slice ready flag.

- #define ASIC_STATUS_LINE_BUFFER_DONE 0x080

  The bit mask for the flag, which indicates whether a line buffer is used.

- #define ASIC_STATUS_POLL_SLICEINFO_TIMEOUT 0x2000

  The bit mask for the flag, which indicates whether the slice information is updated in time.

- #define ASIC_STATUS_UFBC_DEC_ERR 0x4000

  The bit mask for the external frame buffer compressor (UFBC) status flag, which indicates whether an decode error occurs when UFBC is used to fetch the input picture data.

- #define ASIC_STATUS_UFBC_CFG_ERR 0x8000

  The bit mask for the external frame buffer compressor (UFBC) status flag, which indicates whether an config error occurs when UFBC is used to fetch the input picture data.

- #define ASIC_STATUS_SBI_OUT_OF_SYNC 0x20000

  The bit mask for SBI out of sync error.

- #define ASIC_STATUS_SBI_TIMEOUT 0x10000

  The bit mask for SBI timeout error.

- #define ASIC_STATUS_HW_TIMEOUT 0x040

  The bit mask for the hardware timeout flag.

- #define ASIC_STATUS_BUFF_FULL 0x020

  The bit mask for the flag, which indicates the stream buffer is full.

- #define ASIC_STATUS_HW_RESET 0x010

  The bit mask for the flag, which indicates the hardware reset is completed.

- #define ASIC_STATUS_ERROR 0x008

  The bit mask for the bus error flag.

- #define ASIC_STATUS_FRAME_READY 0x004

  The bit mask for the flag, which indicates whether the encoding of one frame is completed.

- #define ASIC_IRQ_LINE 0x001

  The bit mask for the flag, which indicates whether IRQ occurs.

- #define ASIC_STATUS_ALL

  The bit mask for valid status flags. For details, see Section Macro Definition.

- #define EWL_OK 0

  API runs successfully.

- #define EWL_ERROR -1

  Errors occur when executing the function, such as the specified feature is not supported.

- #define EWL_NOT_SUPPORT -2

  The function execution fails, because the specified feature is not supported.

- #define EWL_HW_WAIT_OK EWL_OK

  The hardware is enabled properly even if an error is detected.

- #define EWL_HW_WAIT_ERROR EWL_ERROR

  An error is detected by the software.

- #define EWL_HW_WAIT_TIMEOUT 1

  The hardware times out.

- #define EWL_HW_BUS_TYPE_UNKNOWN 0

  The bus type is unset.

- #define EWL_HW_BUS_TYPE_AHB 1

  The bus type is AHB.

- #define EWL_HW_BUS_TYPE_OCP 2

  The bus type is OCP.

- #define EWL_HW_BUS_TYPE_AXI 3

  The bus type is AXI.

- #define EWL_HW_BUS_TYPE_PCI 4

  The bus type is PCI.

- #define EWL_HW_BUS_WIDTH_UNKNOWN 0

  The bus width is unset.

- #define EWL_HW_BUS_WIDTH_32BITS 1

  The bus width is 32 bits.

- #define EWL_HW_BUS_WIDTH_64BITS 2

  The bus width is 64 bits.

- #define EWL_HW_BUS_WIDTH_128BITS 3

  The bus width is 128 bits.

- #define EWL_HW_SYNTHESIS_LANGUAGE_UNKNOWN 0

  The synthesis language is unset.

- #define EWL_HW_SYNTHESIS_LANGUAGE_VHDL 1

  The synthesis language is VHDL.

- #define EWL_HW_SYNTHESIS_LANGUAGE_VERILOG 2

  The synthesis language is VeriLog.

- #define EWL_HW_CONFIG_NOT_SUPPORTED 0

  The current hardware configuration does not support the specified feature.

- #define EWL_HW_CONFIG_ENABLED 1

  The current hardware configuration supports the specified feature.

- #define EWL_MAX_CORES 4

  The supported maximum number of cores.

- #define CORE(id) ((u32)(id)&0xff)

  Obtains the core index from the core information words.
  core_id[0:7]: The core index.

- #define NODE(id) ((u32)(id) >> 16)

  Obtains the slice node index from the core information words.
  core_id[16:23]: The slice node index.

- #define COREID(node, core) (((u32)(node) << 16) | ((u32)(core)&0xff))

  The derived core ID, which is the combination of the slice node index and the core index.

5.1.1 Marco Definition

5.1.1.1 HW_PRODUCT_SYSTEM60

```
#define HW_PRODUCT_SYSTEM60(
                x )
```

Value:

```
  ((HW_ID_PRODUCT_VC8000E == HW_ID_PRODUCT(x)) && \
   (HW_ID_MAJOR_NUMBER(x) == 0x60))
```

5.1.1.2 ASIC_STATUS_ALL

```
#define ASIC_STATUS_ALL
```

Value:

```
  (ASIC_STATUS_SEGMENT_READY | ASIC_STATUS_FUSE_ERROR                    \
   | ASIC_STATUS_SLICE_READY | ASIC_STATUS_LINE_BUFFER_DONE                \
   | ASIC_STATUS_HW_TIMEOUT | ASIC_STATUS_BUFF_FULL | ASIC_STATUS_HW_RESET  \
   | ASIC_STATUS_ERROR | ASIC_STATUS_FRAME_READY | ASIC_STATUS_UFBC_DEC_ERR \
   | ASIC_STATUS_SBI_OUT_OF_SYNC | ASIC_STATUS_SBI_TIMEOUT                 \
   | ASIC_STATUS_UFBC_CFG_ERR | ASIC_STATUS_POLL_SLICEINFO_TIMEOUT)
```

5.1.2 Functions

5.1.2.1 EWLReadAsicID()

```
u32 EWLReadAsicID (
            u32 core_id,
            const void * ctx )
```

Obtains the hardware ID of the specified core.

Parameters

in	*core_id*	The ID to identify the core.
in	*ctx*	The context used to access the hardware, for example, the device description for this instance.

Returns

The hardware ID of the specified core.

5.1.2.2 EWLGetCoreNum()

```
u32 EWLGetCoreNum (
            const void * ctx )
```

Obtains the core number.

Parameters

in	*ctx*	The context used to access the hardware, for example, the device description for this instance.

Returns

The core number in the system.

5.1.2.3 EWLCheckCutreeValid()

```
i32 EWLCheckCutreeValid (
            const void * inst )
```

Specifies whether the CuTree hardware is available in the system.

Parameters

in	*inst*	An EWL instance.

Returns

EWL_ERROR

EWL_OK

5.1.2.4 EWLGetDec400Attribute()

```
void EWLGetDec400Attribute (
            u32 * tile_size,
            u32 * bits_tile_in_table,
            u32 * planar420_cbcr_table_style )
```

Obtains the DEC400 attribute for specified input pixel format.

Parameters

in	*pixel_format*	The picture pixel format, which is compressed by DEC400, in the buffer.
out	*tile_size*	A pointer to the tile size of the corresponding pixel format.
out	*bits_tile_in_table*	A pointer to the bits used in the table for one tile.
out	*planar420_cbcr_table_style*	A pointer to the chroma table attribute.

5.1.2.5 EWLReadAsicConfig()

```
const EWLHwConfig_t* EWLReadAsicConfig (
            u32 core_id,
            const void * ctx )
```

Obtains hardware configuration information.

Parameters

in	*core_id*	The ID to identify the core.
in	*ctx*	The context used to access the hardware, for example, the device description for this instance.

Returns

The hardware configuration of the specified core.

5.2 Memory Management

Marco Definition

- #define MEM_CHUNKS 720

  The maximum number of linear memory used for status only.
- #define LINMEM_ALIGN 4096

  The page size and alignment for linear memory chunks.
- #define NEXT_ALIGNED(x) ((((ptr_t)(x)) + LINMEM_ALIGN - 1) / LINMEM_ALIGN * LINMEM_ALIGN)

  The nearest aligned linear memory address before the input address x.
- #define NEXT_ALIGNED_SYS(x, align) ((((ptr_t)(x)) + align - 1) / align * align)

  The nearest address aligned with "align" before the input address x.
- #define EWL_MEM_TYPE_CPU 0x0000U

  The memory accessed by CPU. It is a non-secure memory.
- #define EWL_MEM_TYPE_SLICE 0x0001U

  The output buffer for the encoder.

- #define EWL_MEM_TYPE_DPB 0x0002U

  The input buffer or reconstructed frame buffer for the encoder.
- #define EWL_MEM_TYPE_VPU_WORKING 0x0003U

  The working buffer for the encoder. It is a non-secure memory.
- #define EWL_MEM_TYPE_VPU_WORKING_SPECIAL 0x0004U

  VPU reads the memory; CPU reads and writes the memory.
- #define EWL_MEM_TYPE_VPU_ONLY 0x0005U

  VPU reads and writes the memory.
- #define EWL_MEM_USAGE_HINT_MASK 0x7F0000U

  The bit mask (bits [22:16]) that indicates the usage hint of the buffer.
- #define EWL_MEM_USAGE_HINT_SHIFT 16

  The least significant bit (LSB) of memory usage hint bits in memory attribute words.
- #define EWL_MEM_ATTR_SHIFT 23

  Bit 23, which indicates whether the memory attribute is META or CORE.
- #define ATTR_NOT_DEF META

  Non-specified attributes of a memory. These attribute are regarded as META.
- #define SET_MEM_USAGE(mem_type, usage, is_secure)

  Sets the memory usage hint and security flag in memory attribute words. For details, see Section Macro Definition.
- #define EWLGetMemAttribute(mem_type) ((mem_type>>EWL_MEM_ATTR_SHIFT)&0x1)

  Specifies whether the memory is secure by parsing the memory attribute flag.
- #define CPU_RD 0x0100U

  CPU reads the memory.
- #define CPU_WR 0x0200U

  CPU writes the memory.
- #define VPU_RD 0x0400U

  VPU reads the memory.
- #define VPU_WR 0x0800U

  VPU writes the memory.
- #define EXT_RD 0x1000U

  External IP reads the memory.
- #define EXT_WR 0x2000U

  External IP writes the memory.
- #define EWL_PAR_ERROR EWL_ERROR

  The provided parameter is wrong.
- #define EWL_HW_ERROR -2

  Errors occur, when the specified function runs.

5.2.1 Marco Definition

5.2.1.1 SET_MEM_USAGE

```
#define SET_MEM_USAGE(
                mem_type,
                usage,
                is_secure )
```

Value:

```
    do { \
    (mem_type) |= ((usage)«EWL_MEM_USAGE_HINT_SHIFT | \
    ((USAGE_ATTR(usage)&is_secure)«EWL_MEM_ATTR_SHIFT)); \
} while(0)
```

Parameters

in,out	*mem_type*	Input: The memory attribute words and other attributes. Output: The input data added with usage hint and security flag.
in	*usage*	The memory usage hint.
in	*is_secure*	The security status of the memory.

5.2.2 Enumeration Type

5.2.2.1 MEM_ATTR

```
enum MEM_ATTR
```

The security attribute of a buffer.

Enumerator

META	The buffer does not have sensitive data.
CORE	The buffer has sensitive data.

5.2.2.2 EWLMemUsageHint

```
enum EWLMemUsageHint
```

Specifies the usage of a buffer.

Enumerator

EWL_MEM_USAGE_UNDEFINED	(Default) The attribute is not defined.
EWL_MEM_USAGE_IN_SURFACE	The attribute for an input picture buffer.
EWL_MEM_USAGE_IN_QPMAP	The attribute for a QP map buffer.
EWL_MEM_USAGE_IN_OVERLAY	The attribute for a buffer that stores overlay pictures.
EWL_MEM_USAGE_IN_CUCTLMAP	The attribute for a CU control buffer.
EWL_MEM_USAGE_IN_CUIDXMAP	(Reserved) The attribute for a CU index map buffer.
EWL_MEM_USAGE_IN_DOWNSACLE	The attribute for downscale input for pass-1 encoding when lookahead is enabled.
EWL_MEM_USAGE_IN_SURFACE_DECTS	The attribute for DEC400 tile status of input picture buffer.
EWL_MEM_USAGE_IN_OVERLAY_DECTS	The attribute for DEC400 tile status of input overlay buffer.
EWL_MEM_USAGE_IN_MVINFO	(Reserved) The attribute for MV information.
EWL_MEM_USAGE_IN_TRANSFORM	(For test only) The attribute for input picture translation.
EWL_MEM_USAGE_IN_LINEBUF	The attribute for a line buffer (as input picture) in low-latency mode.
EWL_MEM_USAGE_IN_OSDMAP	The attribute for OSD map buffer.
EWL_MEM_USAGE_OUT_STRM	The attribute for output stream.
EWL_MEM_USAGE_OUT_CUINFO	The attribute for output CU information.
EWL_MEM_USAGE_OUT_SIZETABLE	The attribute for the buffer that stores the NAL unit size of output streams.
EWL_MEM_USAGE_OUT_CTBBITS	The attribute for status buffer which have bits count of every CTB.
EWL_MEM_USAGE_OUT_SCAL	The attribute for downscale output picture.

Enumerator

EWL_MEM_USAGE_TMP_RFCTS	The attribute for reference compression tables.
EWL_MEM_USAGE_TMP_COEFF	The attribute for scratch of an entropy engine.
EWL_MEM_USAGE_TMP_TILECTX	The attribute for scratch of tile context.
EWL_MEM_USAGE_TMP_TILEHEIGHT	The attribute for scratch of tile edge.
EWL_MEM_USAGE_TMP_RECON	The attribute for a buffer that stores reconstructed frames.
EWL_MEM_USAGE_TMP_CTBRC	The attribute for a buffer used for CTBRC.
EWL_MEM_USAGE_TMP_MCSYNC	The attribute for multi-core synchronization words.
EWL_MEM_USAGE_TMP_TMVP	The attribute for a TMVP buffer.
EWL_MEM_USAGE_TMP_COST	The attribute for a cost buffer.
EWL_MEM_USAGE_TMP_EXTSRAM	The attribute for the external SRAM of ME.
EWL_MEM_USAGE_TMP_H264COL	The attribute for colocate information for H264.
EWL_MEM_USAGE_TMP_AV1FRMCTX	The attribute for frame context for AV1.
EWL_MEM_USAGE_TMP_AV1PRC	The attribute for pre-carry buffer for AV1.
EWL_MEM_USAGE_TMP_VP9FRMCTX	The attribute for frame context for VP9.
EWL_MEM_USAGE_TMP_FRAMEINFO	The attribute for frame status output.
EWL_MEM_USAGE_TMP_SLICEINFO	The attribute for the slice information used in low-latency encoding DDR mode.

5.2.2.3 EWLMemSyncDirection

```
enum EWLMemSyncDirection
```

Specifies the data transfer direction of EWL.

Enumerator

HOST_TO_DEVICE	Copies data from the host memory to the device memory.
DEVICE_TO_HOST	Copies data from the device memory to the host memory.

5.2.3 Functions

5.2.3.1 EWLMallocRefFrm()

```
i32 EWLMallocRefFrm (
            const void * instance,
            u32 size,
            u32 alignment,
            EWLLinearMem_t * info )
```

Allocates a linear memory for reference frames. Reference frames are only accessed by hardware.

Parameters

in	*size*	The bytes to be allocated.
in	*alignment*	The required alignment of the buffer.
out	*info*	The information of the memory allocated as a linear buffer.

5.2.3.2 EWLFreeRefFrm()

```
void EWLFreeRefFrm (
            const void * inst,
            EWLLinearMem_t * info )
```

Releases the linear buffer allocated by EWLMallocRefFrm().

Parameters

in	*info*	The information to identify the buffer to be released.

5.2.3.3 EWLMallocLinear()

```
i32 EWLMallocLinear (
            const void * instance,
            u32 size,
            u32 alignment,
            EWLLinearMem_t * info )
```

Specifies software/hardware-shared buffer without memory synchronization.

When MMU is disabled, this buffer should be physically continuous.

Parameters

in	*size*	The bytes to be allocated.
in	*alignment*	The required alignment of the buffer.
out	*info*	The information of the memory allocated as a linear buffer.

5.2.3.4 EWLFreeLinear()

```
void EWLFreeLinear (
            const void * inst,
            EWLLinearMem_t * info )
```

Releases the linear memory allocated by `EWLMallocLinear()`.

Parameters

in	*info*	The information to identify the buffer to be released.

5.2.3.5 EWLmalloc()

```
void* EWLmalloc (
            u32 n )
```

Allocates n bytes and returns a pointer to the allocated memory.

This function is a wrapper of `malloc()`.

Parameters

in	*n*	The number of bytes to be allocated.

Returns

The address of the allocated memory.

5.2.3.6 EWLcalloc()

```
void* EWLcalloc (
            u32 n,
            u32 s )
```

Allocates memory for an array of n elements of s bytes each and returns a pointer to the allocated memory. The memory is set to zero.

This function is a wrapper of `calloc()`.

Parameters

in	*n*	The number of elements.
in	*s*	The size of each element, in bytes.

Returns

The address of the allocated memory.

5.2.3.7 EWLfree()

```
void EWLfree (
            void * p )
```

Releases the allocated memory.

This function is a wrapper of `free()`.

5.2.3.8 EWLmemcpy()

```
mem_ret EWLmemcpy (
            void * d,
            const void * s,
            u32 n )
```

Copies n bytes from memory area s to memory area d.

This function is a wrapper of `memcpy()`.

Parameters

in	*d*	The destined memory area for the copied bytes.
in	*s*	The source memory area where data values are copied.
in	*n*	The size of the copied memory area, in bytes.

5.2.3.9 EWLmemset()

```
mem_ret EWLmemset (
            void * d,
            i32 c,
            u32 n )
```

Fills memory with a constant byte.

This function is a wrapper of `memset()`.

Parameters

in	*d*	The start memory address where the value is filled.
in	*c*	The byte value to be filled.
in	*n*	The number of bytes to be filled.

5.2.3.10 EWLmemcmp()

```
int EWLmemcmp (
            const void * s1,
            const void * s2,
            u32 n )
```

Compares two memory areas.

This function is a wrapper of `memcmp()`.

Parameters

in	*s1*	One memory area to be compared.
in	*s2*	Another memory area to be compared.
in	*n*	The size of s1 and s2, in bytes.

5.2.3.11 EWLGetLineBufSram()

```
i32 EWLGetLineBufSram (
            const void * instance,
            EWLLinearMem_t * info )
```

Obtains the information, including address and size, of the input line buffer.

Parameters

in	*info*	Information to identify the input line buffer.

5.2.3.12 EWLMallocLoopbackLineBuf()

```
i32 EWLMallocLoopbackLineBuf (
```

```
            const void * instance,
            u32 size,
            EWLLinearMem_t * info )
```

Allocates memory for the loopback line buffer.

Parameters

in	*size*	The bytes to be allocated.
out	*info*	Information to identify the loopback line buffer.

5.2.3.13 EWLSyncMemData()

```
i32 EWLSyncMemData (
            EWLLinearMem_t * mem,
            u32 offset,
            u32 length,
            enum EWLMemSyncDirection dir )
```

Transfers data between the host and device.

This function is available only if memory synchronization is supported.

Parameters

in	*mem*	The buffer information used for synchronization.
in	*offset*	The offset related to the hardware register base address, in bytes.
in	*length*	The number of bytes to be synchronized.
in	*dir*	The direction of transferring data.

Returns

EWL_OK

EWL_PAR_ERROR

5.2.3.14 EWLMemSyncAllocHostBuffer()

```
i32 EWLMemSyncAllocHostBuffer (
            const void * inst,
            u32 size,
            u32 alignment,
            EWLLinearMem_t * buff )
```

Allocates memory for a buffer in the host.

This function is available only if memory synchronization is supported.

Parameters

in	*alignment*	The alignment for allocation.
in	*size*	The bytes to be allocated.
out	*buff*	The information of the memory allocated as a buffer in the host.

Returns

EWL_OK

5.2.3.15 EWLMemSyncFreeHostBuffer()

```
i32 EWLMemSyncFreeHostBuffer (
            const void * inst,
            EWLLinearMem_t * buff )
```

Releases the memory of a buffer in the host.

This function is available only if memory synchronization is supported.

Parameters

in	*buff*	The information of the memory to be released.

Returns

EWL_OK

5.3 EWL API

Marco Definition

- #define EWL_IS_HEVC_CLIENT(clientType) ((clientType) == EWL_CLIENT_TYPE_HEVC_ENC)

  Specifies whether the client type is HEVC.
- #define EWL_IS_H264_CLIENT(clientType) ((clientType) == EWL_CLIENT_TYPE_H264_ENC)

  Specifies whether the client type is H.264.
- #define EWL_IS_AV1_CLIENT(clientType) ((clientType) == EWL_CLIENT_TYPE_AV1_ENC)

  Specifies whether the client type is AV1.
- #define EWL_IS_VP9_CLIENT(clientType) ((clientType) == EWL_CLIENT_TYPE_VP9_ENC)

  Specifies whether the client type is VP9.
- #define EWL_IS_JPEG_CLIENT(clientType) ((clientType) == EWL_CLIENT_TYPE_JPEG_ENC)

  Specifies whether the client type is JPEG.
- #define EWL_IS_CUTREE(clientType) ((clientType) == EWL_CLIENT_TYPE_CUTREE)

  Specifies whether the client type is CuTree.
- #define EWL_IS_VIDEOSTAB(clientType) ((clientType) == EWL_CLIENT_TYPE_VIDEOSTAB)

  Specifies whether the client type is video stabilization.
- #define EWL_IS_VIDEO_CLIENT(clientType)

  Specifies whether the client is for video encoding. For details, see Section Macro Definition.
- #define EWL_SUPPORT_CLIENT(cfg, clientType)

  Specifies whether the given client type is supported and enabled. For details, see Section Macro Definition.
- #define EWL_SUPPORT_CUTREE(cfg, clientType) ((cfg->cuTreeSupport == 1) && EWL_IS_CUTREE(clientType))

  Specifies whether the client is CuTree and whether it is supported.

Typedef

- typedef void(* **CoreWaitCallBackFunc**) (const void *ewl, void *data)
- typedef void(* **CoreWaitCallBackFunc2**) (const void *ewl, void *data, u32 param)

5.3.1 Marco Definition

5.3.1.1 EWL_IS_VIDEO_CLIENT

```
#define EWL_IS_VIDEO_CLIENT(
                clientType )
```

Value:

```
( EWL_IS_HEVC_CLIENT(clientType) \
|| EWL_IS_H264_CLIENT(clientType) \
|| EWL_IS_AV1_CLIENT(clientType) \
|| EWL_IS_VP9_CLIENT(clientType))
```

5.3.1.2 EWL_SUPPORT_CLIENT

```
#define EWL_SUPPORT_CLIENT(
                cfg,
                clientType )
```

Value:

```
(((cfg->hevcEnabled == 1) && EWL_IS_HEVC_CLIENT(clientType)) \
|| ((cfg->h264Enabled == 1) && EWL_IS_H264_CLIENT(clientType)) \
|| ((cfg->av1Enabled == 1) && EWL_IS_AV1_CLIENT(clientType)) \
|| ((cfg->vp9Enabled == 1) && EWL_IS_VP9_CLIENT(clientType)) \
|| ((cfg->jpegEnabled == 1) && EWL_IS_JPEG_CLIENT(clientType)) \
|| ((cfg->vsSupport == 1) && EWL_IS_VIDEOSTAB(clientType)))
```

5.3.2 Enumeration Type

5.3.2.1 CLIENT_TYPE

```
enum CLIENT_TYPE
```

The hardware engine type.

Enumerator

EWL_CLIENT_TYPE_MAIN	Video or image encoder main core
EWL_CLIENT_TYPE_H264_ENC	H.264 encoder
EWL_CLIENT_TYPE_HEVC_ENC	HEVC encoder
EWL_CLIENT_TYPE_VP9_ENC	VP9 encoder
EWL_CLIENT_TYPE_JPEG_ENC	JPEG encoder
EWL_CLIENT_TYPE_CUTREE	CuTree analyzer engine
EWL_CLIENT_TYPE_VIDEOSTAB	(Reserved) Video stabilization engine
EWL_CLIENT_TYPE_DEC400	DEC400 engine
EWL_CLIENT_TYPE_AV1_ENC	AV1 encoder
EWL_CLIENT_TYPE_L2CACHE	L2Cache engine
EWL_CLIENT_TYPE_AXIFE	AXI_FE engine
EWL_CLIENT_TYPE_APBFT	APB filter engine
EWL_CLIENT_TYPE_AXIFE_1	AXI_FE_1 engine
EWL_CLIENT_TYPE_MEM	memalloc
EWL_CLIENT_TYPE_MMU0	MMU0
EWL_CLIENT_TYPE_MMU1	MMU1
EWL_CLIENT_TYPE_UFBC	UFBC

5.3.3 Functions

5.3.3.1 EWLGetDec400Coreid()

```
i32 EWLGetDec400Coreid (
        const void * inst )
```

Obtains the core ID of a core, to which the DEC400 is attached.

Parameters

in	*inst*	An EWL instance.

Returns

The index of the subsystem.

5.3.3.2 EWLGetVcmdVersionId()

```
u32 EWLGetVcmdVersionId (
            const void * inst )
```

Obtains the VCMD hardware version ID.

Parameters

in	*inst*	An EWL instance.

Returns

The version ID of the VCMD.

5.3.3.3 MapAsicRegisters()

```
int MapAsicRegisters (
            void * ewl )
```

Maps the registers for access from user space.

Parameters

in	*ewl*	An EWL instance.

Returns

`0`: Mapping succeeds.

`-1`: Mapping fails.

5.3.3.4 EWLGetClientType()

```
u32 EWLGetClientType (
            const void * inst )
```

Obtains the client type used to initialize the instance.

Parameters

in	*inst*	An EWL instance.

Returns

The client type.

5.3.3.5 EWLGetCoreTypeByClientType()

```
u32 EWLGetCoreTypeByClientType (
            u32 client_type )
```

Converts the client type to core type.

Parameters

in	*client_type*	The client type to write.

Returns

The core type.

5.3.3.6 EWLInit()

```
const void* EWLInit (
            EWLInitParam_t * param )
```

Initializes an EWL instance.

Parameters

in	*param*	The configuration for this instance.

Returns

An EWL instance, or NULL in case of function failure.

5.3.3.7 EWLRelease()

```
i32 EWLRelease (
            const void * inst )
```

Releases an EWL instance.

Returns

EWL_OK

EWL_ERROR

5.3.3.8 EWLReserveHw()

```
i32 EWLReserveHw (
            const void * inst,
            u32 * core_info,
            u32 * job_id )
```

Reserves the hardware resources for the current EWL instance.

This function should be called after the software prepared a buffer and register values for encoding one frame and the hardware can start encoding. It is blocked if resources are unavailable. After the hardware resources are reserved, the software can access these resources directly.

Parameters

in,out	*core_info*	The information to specify which core or cores can be used for the current encoding.
out	*job_id*	The ID used to note that the current frame encoding job is filled.

Returns

EWL_OK

EWL_ERROR

5.3.3.9 EWLReleaseHw()

```
void EWLReleaseHw (
            const void * inst )
```

Releases the hardware resources.

This function should be called after the hardware finishes encoding one frame and the software has obtained the encoding information from hardware registers. After calling this function, the software releases the hardware resources reserved for the current instance. Then, other instances can access these hardware resources.

The software can only access hardware resources, such as registers, between calling EWLReserveHw() and EWLReleaseHw().

5.3.3.10 EWLGetPerformance()

```
u32 EWLGetPerformance (
            const void * inst )
```

Obtains the number of hardware cycles used for encoding one frame.

The cycles are counted according to the encoder clock.

Returns

The number of hardware cycles used for encoding one frame.

5.3.3.11 EWLWriteReg()

```
void EWLWriteReg (
            const void * inst,
            u32 offset,
            u32 val )
```

Writes a value to a hardware register.

All registers are written before the hardware is enabled for encoding.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.
in	*val*	The value to be written into the register.

5.3.3.12 EWLWriteBackReg()

```
void EWLWriteBackReg (
            const void * inst,
            u32 offset,
            u32 val )
```

Writes back a value to a hardware register.

This function is called when a callback is completed, or a frame encoding in a multi-core scenario is completed.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.
in	*val*	The value to be written into the register.

5.3.3.13 EWLWriteCoreReg()

```
void EWLWriteCoreReg (
            const void * inst,
            u32 offset,
            u32 val,
            u32 core_id )
```

Writes a value to a hardware register by the specified core ID.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.
in	*val*	The value to be written into the register.
in	*core_id*	The ID to specify the core to write.

5.3.3.14 EWLWriteCoreRegByVcmd()

```
void EWLWriteCoreRegByVcmd (
            const void * inst,
            u32 offset,
            u32 num,
            u32 * val )
```

Writes values to hardware registers in vcmd mode.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.
in	*val*	The values to be written into the registers.
in	*num*	The register number to write.

5.3.3.15 EWLWriteRegbyClientType()

```
void EWLWriteRegbyClientType (
            const void * inst,
            u32 offset,
            u32 val,
            u32 client_type )
```

Writes a value to a hardware register by the specified client type.

This function is often used to access peripheral registers of sub-IPs.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.
in	*val*	The value to be written into the register.
in	*client_type*	The client type to write.

5.3.3.16 EWLWriteBackRegbyClientType()

```
void EWLWriteBackRegbyClientType (
            const void * inst,
            u32 offset,
            u32 val,
            u32 client_type )
```

Writes back a value to a hardware register by the specified client type.

This function is often used to access peripheral registers of sub-IPs.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.
in	*val*	The value to be written into the register.
in	*client_type*	The client type to write.

5.3.3.17 EWLReadReg()

```
u32 EWLReadReg (
            const void * inst,
            u32 offset )
```

Obtains the value in a status register.

Such registers are updated by the hardware and are read after an interrupt is generated.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.

Returns

The value of the hardware register.

5.3.3.18 EWLReadRegbyClientType()

```
u32 EWLReadRegbyClientType (
            const void * inst,
            u32 offset,
            u32 client_type )
```

Obtains the value in a status register of one client.

Such registers are updated by the hardware and are read after an interrupt is generated.

This function is often used to access peripheral registers of sub-IPs.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes.

Returns

The value of the hardware register.

5.3.3.19 EWLEnableHW()

```
i32 EWLEnableHW (
            const void * inst,
            u32 offset,
            u32 val )
```

Enables the hardware to start encoding a frame.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes. Its value should be set to 0x14.
in	*val*	The value to be written into the register. Bit 0 of this parameter should be 1.

5.3.3.20 EWLDisableHW()

```
void EWLDisableHW (
            const void * inst,
            u32 offset,
            u32 val )
```

Disables the hardware and stops encoding.

Parameters

in	*offset*	The offset related to the hardware register base address, in bytes. Its value should be set to 0x14.
in	*val*	The value to be written into the register. Bit 0 of this parameter should be 0.

5.3.3.21 EWLWaitHwRdy()

```
i32 EWLWaitHwRdy (
            const void * instance,
            u32 * slicesReady,
            void * waitOut,
            u32 * status_register )
```

Waits for the hardware to complete encoding.

After the hardware is enabled, this function is called to wait for frame encoding completion or error interrupts. In polling mode, the software polls the hardware status register to check whether there is any hardware status change. In interrupt mode, the software is in "sleep" status and waits for IRQ. The final software product works in interrupt mode.

Parameters

out	*slicesReady*	The value saved in this pointer contains the number of encoded slices in the hardware output buffer.
out	*status_register*	The value saved in this pointer contains the bit map, which indicates the status of the encoding work.

Returns

EWL_HW_WAIT_OK
EWL_HW_WAIT_ERROR
EWL_HW_WAIT_TIMEOUT

5.3.3.22 EWLGetClientOffset()

```
u16 EWLGetClientOffset (
            const void * inst,
            u16 client_type )
```

Obtains the register offset of a hardware engine in the subsystem.

Parameters

inst	An EWL instance.
client_type	The hardware engine.

Returns

A value equal to 0xffff: the hardware engine inquired is not available.

A value smaller than 0xffff: the inquired hardware engine is detected and the value equals the register offset of the hardware engine, in bytes.

5.3.3.23 EWLGetClientVcmdStatusBufOffset()

```
u16 EWLGetClientVcmdStatusBufOffset (
            const void * inst,
            u16 comp_type )
```

Obtains the submodule register offset of status cmdbuf in the subsystem.

Parameters

inst	An EWL instance.
client_type	The hardware engine.

Returns

A value equal to 0xffff: the hardware engine inquired is not available.

A value smaller than 0xffff: the inquired hardware engine is detected and the value equals the submodule register offset of status cmdbuf, in bytes.

5.3.3.24 EWLReadRegInit()

```
u32 EWLReadRegInit (
```

```
            const void * inst,
            u32 offset )
```

Obtains the read-only registers loaded by VCMD during initialization.

Parameters

in	*offset*	The offset related to the hardware register base address.

Returns

`EWL_OK` when VCMD is disabled.

The register values obtained during the VCMD initialization.

5.3.3.25 EWLReserveCmdbuf()

```
i32 EWLReserveCmdbuf (
            const void * inst,
            EWLResource_t * data )
```

Reserves one VCMD buffer for the current instance.

This function requests a VCMD buffer from a system such as the kernel driver. When this function returns, the software fills the buffer with commands to set up the encoding parameters for frame encoding. This function waits internally untill there is a free VCMD buffer available.

Parameters

in,out	*data*	• The `data->vcmdbuf.size` is an input parameter indicating the size required by one VCMD buffer. • The `data->vcmdbuf.id` is filled with the ID of the available VCMD buffer. • The `data->vcmdbuf.status_ba` is filled with the address where the register values read from the hardware are stored.

Returns

`EWL_OK`

`EWL_ERROR`

5.3.3.26 EWLLinkRunCmdbuf()

```
i32 EWLLinkRunCmdbuf (
            const void * inst,
            u16 cmdbufid,
            u16 cmdbuf_size )
```

Puts a VCMD buffer into queue.

When a queue is empty, the VCMD buffer is run at once. Otherwise, the VCMD buffer is linked to the end of the queue.

Parameters

in	*cmdbufid*	The ID to identify the VCMD buffer.
in	*cmdbuf_size*	The size of the VCMD buffer.

Returns

EWL_ERROR

EWL_OK

5.3.3.27 EWLWaitCmdbuf()

```
i32 EWLWaitCmdbuf (
            const void * inst,
            u16 cmdbufid,
            u32 * status )
```

Waits until all commands in the VCMD buffer obtained by EWLReserveCmdbuf() are executed.

After the VCMD buffer is put into the VCMD buffer queue or run directly by calling EWLLinkRunCmdbuf(), this function is called to wait for all commands being executed. Such waiting can be asynchronous, awakened by the IRQ triggered by a VCMD interrupt. It can also be synchronous to poll the VCMD status register.

Parameters

in	*cmdbufid*	The ID to identify the VCMD buffer to run.
out	*status*	The value stored in this pointer contains the bit map, which indicates the status of the encoding job.
out	*slices_rdy*	The value that indicates how many encoded slices are filled to this address.

Returns

EWL_OK

EWL_ERROR

EWL_HW_WAIT_ERROR

5.3.3.28 EWLGetRegsByCmdbuf()

```
void EWLGetRegsByCmdbuf (
            const void * ewl,
            u16 cmdbufid,
            u32 * regMirror )
```

Copies register values from a VCMD status buffer to mirror registers.

Parameters

in	*cmdbufid*	The ID to identify the VCMD buffer.
out	*regMirror*	The address of a buffer to save the register values.

5.3.3.29 EWLReleaseCmdbuf()

```
i32 EWLReleaseCmdbuf (
            const void * inst,
            u16 cmdbufid )
```

Releases the VCMD buffer used for encoding the current frame.

This function should be called after the hardware finishes processing one VCMD Buffer and software has gotten the encoding information from VCMD status buffer. After calling this API, The VCMD buffer reserved for currrent instance will be returned to system. Then other instance can use it. Software can only access the VCMD buffer between `EWLReserveCmdbuf()` and `EWLReleaseCmdbuf()`.

Parameters

in	*cmdbufid*	The ID to identify the VCMD buffer used for encoding.

5.3.3.30 EWLSetReserveBaseData()

```
void EWLSetReserveBaseData (
            const void * inst,
            u64 interrupt_ctrl,
            u32 client_type )
```

Sets the information about the commands stored in the VCMD Buffer.

This function is used in VCMD mode only. The information helps the driver estimate the workload and schedule the job in the VCMD Buffer. In parmeter interrupt_ctrl, bit[31] is a flag used to indicate mode.

- When mode is 0, interrupt is generated adaptively. In such mode,
  - bit[30:0] is the estimating executing time for current job.
- When mode flag is 1, interrupt is generated according to bit[0].
  - When bit[0] is 0, VCMD will not generate interrupt when run JMP command;
  - when bit[0] is 1, VCMD will generate interrupt when run JMP command; bit[39:32] is batch count.

Parameters

in	*interrupt_ctrl*	control how to generate interrupt when execute VCMD JMP command.
in	*client_type*	The hardware engine type.

5.3.3.31 EWLGetVCMDSupport()

```
u32 EWLGetVCMDSupport (
            const void * inst )
```

Specifies whether VCMD is enabled.

Returns

1: Enabled.

0: Disabled.

6 Data Structure Documentation

6.1 CacheData_t

The L2Cache.

Data Fields

- void ** cache

  The L2Cache.

6.2 EWLCoreWait_t

Parameters for jobs to be processed in a thread.

Data Fields

- struct queue jobs

  The queue of unfinished jobs, including jobs to be processed and jobs under processing.
- pthread_mutex_t job_mutex

  The mutex for protecting job_pool.
- pthread_cond_t job_cond

  The condition variable for job_mutex.
- struct queue out

  The queue of all completed jobs, including abnormal and normal ones.
- pthread_mutex_t out_mutex

  The mutex for protecting the out queue.

- pthread_cond_t out_cond

  The condition variable for out_mutex.

- pthread_t * tid_CoreWait

  The thread.

- bool bFlush

  Whether the last job is queued.
  0: No.
  1: Yes.

- u32 refer_counter

  The reference frame counter for encoding types of HEVC, H264, AV1, and VP9.
  When the EWL instance is released, the field value equals 0.

- struct queue job_pool

  The pool that stores jobs.

6.3 EWLCoreWaitJob_t

The information of a job

Data Fields

- struct node * next

  The next job.

- u32 id

  The ID of the job the structure describes.

- u32 core_id

  The index of the hardware subsystem.

- const void * inst

  The `vcenc_instance` structure.

- u32 VCE_reg [ASIC_SWREG_AMOUNT]

  All hardware registers. `ASIC_SWREG_AMOUNT` indicates the maximum number of hardware registers.

- i32 out_status

  The interrupt status of the job.
  The value can be:
  - ASIC_STATUS_SEGMENT_READY
  - ASIC_STATUS_FUSE_ERROR
  - ASIC_STATUS_SLICE_READY
  - ASIC_STATUS_LINE_BUFFER_DONE

- *ASIC_STATUS_HW_TIMEOUT*
- *ASIC_STATUS_BUFF_FULL*
- *ASIC_STATUS_HW_RESET*
- *ASIC_STATUS_ERROR*
- *ASIC_STATUS_FRAME_READY.*

- i32 out_poll_sliceinfo_status

  The status indicating whether timeout occurs when polling sliceinfo.

- i32 out_ufbc_status

  the interrupt status of ufbc, mainly for ASIC_STATUS_UFBC_DEC_ERR

- u32 slices_rdy

  The number of completed slices.

- u32 low_latency_rd

  The number of CTB rows that the encoder fetches from the input buffer for the job.

- u32 dec400_enable

  The status of DEC400.
  1: Bypassed.
  2: Enabled.

- CoreWaitCallBackFunc dec400_callback

  (Reserved) The DEC400 callback function.

- void * dec400_data

  Pointer to the DEC400 data.

- u32 axife_enable

  The status of AXI_FE.
  0: Disabled.
  1: Enabled.
  2: Bypassed.
  3: Security mode.

- CoreWaitCallBackFunc axife_callback

  The AXI_FE callback function.

- u32 l2cache_enable

  The status of L2Cache.
  0: Disabled.
  1: Enabled.

- CacheData_t l2cache_data

  The L2Cache.

- CoreWaitCallBackFunc l2cache_callback

  The L2Cache callback function.

- u32 ufbcMode

  The core mode of UFBC.
  0: UFBC disable.
  1: Use the core of AFBC version 0.(only support 32x8 superblock).

2: Use the core of AFBC version 1.(Support 32x8 and 16x16 superblock).
3: Use the core of Dec400.
4: Use the core of PVRIC.

- CoreWaitCallBackFunc2 ufbc_callback

  The UFBC callback. See `EncUfbcAsicStop`.

6.4 EWLHwConfig_t

The hardware configuration information.

Data Fields

- u32 hw_asic_id

  The ASIC_ID to identify the hardware version.
- u32 hw_build_id

  The BUILD_ID of the hardware.
- u32 hevcEnabled

  Whether the hardware supports HEVC encoding.
  0: Does not support.
  1: Supports.
- u32 h264Enabled

  Whether the hardware supports H.264 encoding.
  0: Does not support.
  1: Supports.
- u32 jpegEnabled

  Whether the hardware supports JPEG encoding.
  0: Does not support.
  1: Supports.
- u32 vp9Enabled

  Whether the hardware supports VP9 encoding.
  0: Does not support.
  1: Supports.
- u32 av1Enabled

  Whether the hardware supports AV1 encoding.
  0: Does not support.
  1: Supports.
- u32 cuTreeSupport

  Whether the hardware supports CuTree look-ahead.
  0: Does not support.
  1: Supports.

- u32 maxEncodedWidthHEVC

  The maximum video width supported by the hardware for HEVC encoding, in pixels.

- u32 maxEncodedWidthH264

  The maximum video width supported by the hardware for H.264 encoding, in pixels.

- u32 maxEncodedWidthAV1

  The maximum video width supported by the hardware for AV1 encoding, in pixels.

- u32 maxEncodedWidthVP9

  The maximum video width supported by the hardware for VP9 encoding, in pixels.

- u32 videoHeightExt

  The maximum video height.
  0: 8192 pixels.
  1: 8640 pixels.

- u32 maxEncodedWidthJPEG

  The maximum video width supported by the hardware for JPEG encoding, in pixels.

- u32 bFrameEnabled

  Whether the hardware supports B-frame.
  0: Does not support.
  1: Supports.

- u32 main10Enabled

  Whether the hardware supports main 10 or main 8.
  0: Supports main 8.
  1: Supports main 10.

- u32 RDOQSupportH264

  Whether the hardware supports H.264 rate distorted optimized quantization (RDOQ).
  0: Does not support.
  1: Supports.

- u32 RDOQSupportHEVC

  Whether the hardware supports HEVC RDOQ.
  0: Does not support.
  1: Supports.

- u32 RDOQSupportAV1

  Whether the hardware supports AV1 RDOQ.
  0: Does not support.
  1: Supports.

- u32 RDOQSupportVP9

  Whether the hardware supports VP9 RDOQ.
  0: Does not support.
  1: Supports.

- u32 meHorSearchRange

  The horizontal search range of motion estimation (ME).
  0: 128 pixels for P-frame, 64 pixels for B-frame.
  1: 128 pixels for P-frame, 112 pixels for B-frame.

2: 192 pixels for P-frame, 112 pixels for B-frame.
3: 240 pixels for P-frame, 240 pixels for B-frame.
4: 112 pixels for P-frame, 56 pixels for B-frame.
5: 96 pixels for P-frame, 40 pixels for B-frame.

- u32 meVertSearchRangeH264

  The ME vertical search range for H.264. The effective VSR in unit of pixel is H264_VSR∗VIDEO_VSRUnit.

- u32 meVertSearchRangeHEVC

  The ME vertical Search range for HEVC, AV1, and VP9. The effective VSR in unit of pixel is HEVC_VSR∗VIDEO_VSRUnit.

- u32 meVSRUnitSize

  The ME vertical Search range unit. 0:8 pixels, 4: 4pixels, 8: 8pixels.

- u32 ssimSupport

  Whether the hardware supports SSIM calculation for HEVC and H.264.
  0: Does not support.
  1: Supports.

- u32 psnrSupport

  Whether the hardware supports PSNR calculation for HEVC and H.264.
  0: Does not support.
  1: Supports.

- u32 ssimSupportAV1

  Whether the hardware supports SSIM calculation for AV1.
  0: Does not support.
  1: Supports.

- u32 psnrSupportAV1

  Whether the hardware supports PSNR calculation for AV1.
  0: Does not support.
  1: Supports.

- u32 hevcTileSupport

  Whether the hardware supports HEVC tile.
  0: Does not support.
  1: Supports.

- u32 saoSupport

  Whether the hardware supports HEVC SAO.
  0: Does not support.
  1: Supports.

- u32 rfcEnable

  Whether the hardware supports reference frame compressor (RFC).
  0: Does not support.
  1: Supports.

- u32 RFCVersion

  The RFC version to be used.
  0: Luma 64x8, Chroma 2x 8x4 (used from VC8000E)
  1: Luma 8x8, Chroma 2x 8x4 (used by VC8000D)

2: Luma 8x8, Chroma 2x 8x4, with uncompressed region offset at frame top boundary

- u32 roiBlkSize

  the pixel block size controlled by each Qp 0: 8x8 1: 16x16

- u32 roiMapVersion

  The hardware version for ROI map.
  For more information, see Hantro VC9000E Series Memory Buffer and Format Organization.

- u32 cuInforVersion

  The output format of CU statistics information.
  For more information, see Hantro VC9000E Series Memory Buffer and Format Organization.

- u32 cuInfoNoMean

  The mean and variance fields in CuInfo will be invalid (output as 0) if this fuse is 1, otherwise mean and variance will be output normally.

- u32 ctbRcSupport

  Whether the hardware supports CTB-level or MB-level rate control.
  0: Does not support.
  1: Support.

- u32 ctbRcVersion

  The CTB rate control version supported by hardware.
  0: The version only supports to adjust QP for subjective quality.
  1: The version supports to adjust QP for subjective quality and target bits.
  2: The version supports to adjust QP for subjective quality and target bits with a larger range of adjustment.

- u32 frameInfoSupport

  Whether the hardware supports frame-level information output.
  0: Does not support.
  1: Supports.

- u32 progRdoEnable

  Whether the hardware supports programmable RDO level.
  0: Does not support.
  1: Supports.

- u32 ljpegSupport

  Whether the hardware supports JPEG encoding with lossless process.
  0: Does not support.
  1: Supports.

- u32 JpegRoiMapSupport

  Whether the hardware supports ROI map for JPEG encoding.
  0: Does not support.
  1: Supports.

- u32 jpeg400Support

  Whether the hardware supports monochrome JPEG encoding.
  0: Does not support.
  1: Supports.

- u32 jpeg422Support

  Whether the hardware supports YUV422 JPEG encoding.
  0: Does not support.
  1: Supports.

- u32 JpegDHTGenerate

  Whether the hardware supports Huffman table (DHT) generation.
  0: Does not support.
  1: Supports.

- u32 JpegExternalDHT

  Whether the hardware supports user-defined Huffman table for JPEG encoding.
  0: Does not support.
  1: Supports.

- u32 meVertRangeProgramable

  Whether the hardware supports programmable vertical ME range.
  0: Does not support.
  1: Supports.

- u32 MonoChromeSupport

  Whether the hardware supports monochrome H.264 and HEVC encoding.
  0: Does not support.
  1: Supports.

- u32 HevcYUV422Support

  Whether the hardware supports HEVC YUV422.
  0: Does not support.
  1: Supports.

- u32 H264YUV422Support

  Whether the hardware supports H.264 YUV422.
  0: Does not support.
  1: Supports.

- u32 HevcYUV444Support

  Whether the hardware supports HEVC YUV444.
  0: Does not support.
  1: Supports.

- u32 H264YUV444Support

  Whether the hardware supports H.264 YUV444.
  0: Does not support.
  1: Supports.

- u32 HevcTransSkipSupport

  Whether the hardware supports HEVC Transform Skip.
  1: Does not support.
  0: Supports.

- u32 intraTU32Enable

  Whether the hardware supports 32x32 intra TU.
  0: Does not support.
  1: Supports.

- u32 bMultiPassSupport

  Whether the hardware supports look-ahead.
  0: Does not support.
  1: Supports.

- u32 tu32Enable

  Whether the hardware supports 32x32 inter TU.
  0: Does not support.
  1: Supports.

- u32 dynamicMaxTuSize

  Whether the hardware supports dynamic maximum TU size.
  0: Does not support
  1: Supports.

- u32 CtbBitsOutSupport

  Whether the hardware supports number of bits output for each CTB.
  0: Does not support.
  1: Supports.

- u32 encVisualTuneSupport

  Whether the hardware supports 32x32 inter TU.
  0: Does not support.
  1: Supports.

- u32 ctbRcMoreMode

  Whether the hardware supports revert quality adjustment when the subjective CTB rate control is enabled.
  0: Does not support.
  1: Supports.

- u32 deNoiseEnabled

  Whether the hardware supports denoise.
  0: Does not support.
  1: Supports.

- u32 encRef4NInputSupport

  Whether use input down sampled pixel as Ref of ME4N.
  0: Does not support.
  1: Supports.

- u32 encPsyTuneSupport

  Whether the hardware supports tuning quality according to psy-rd.
  0: Does not support.
  1: Supports.

- u32 motionScoreEnable

  Whether the hardware supports motion score calculation and the storage of the result into a software register.
  0: Does not support.
  1: Supports.

- u32 reconSSEsupport

  Whether the hardware supports SSE calculation for recon pixel.

- u32 maxRefNumList0Minus1

The maximum active reference frame number for P-frame encoding.
0: One reference frame.
1: Two reference frames.

- u32 encSliceSizeBits

  The maximum slice height of CTBs or MBs supported by the hardware.
  If the value is n, then the maximum height is 2 to the Power(2,n)-1.

- u32 asymPuSupport

  Whether the hardware supports asymmetric PU partition for HEVC.
  0: Does not support.
  1: Supports.

- u32 maxMergeCand4Support

  Whether the hardware supports the fourth skip candidate for HEVC and H.264.
  0: Does not support.
  1: Supports.

- u32 enMe4nSearch

  Whether the hardware supports ME4N. Without ME4N, ME2N need to do full search (same search range as ME4N) with points jumping.

- u32 searchMe1n3Win

  Whether the hardware supports to search 3 windows to replace search center refine.

- u32 encAQInfoOut

  Whether the hardware supports outputting adaptive quantization information.
  0: Does not support.
  1: Supports.

- u32 prpVersion

  The PRP architecture version: 1: lagecy PRP v1.0 arch 2: new PRP v2.0 arch.

- u32 rgbEnabled

  Whether the hardware supports RGB-to-YUV conversion.
  0: Does not support.
  1: Supports.

- u32 NonRotationSupport

  Whether the hardware supports rotation in pre-processing.
  0: Supports.
  1: Does not support.

- u32 NVFormatOnlySupport

  Whether the hardware only supports NV12 as input.
  0: Supports other format.
  1: Only supports NV12.

- u32 prpLowLatencySupport

  low-latency mechnism version on input picture.
  0: Does not Supports.
  1: support v1
  2:support v2

- u32 prpLowlatencySignalbyDDR

  Pre-Processing use sync word in DDR to do low latency handshake. `prpLowLatencySupport` *should be* `1` *if this feature is selected.*
  0: Supports.
  1: Does not support.
- u32 inLoopDSBilinearSupport

  Whether the hardware supports bilinear on in-loop down-scaler.
  0: Does not support.
  1: Supports.
- u32 inLoopDSRatio

  Whether the hardware supports in-loop down-scaler.
  0: Does not support.
  1: Supports.
- u32 cscExtendSupport

  Whether the hardware supports full or partial range conversion when implementing RGB-to-YUV color conversion.
  0: Does not support.
  1: Supports.
- u32 scalingEnabled

  Whether the hardware supports down-scaling.
  0: Does not support.
  1: Supports.
- u32 scaled420Support

  Whether the hardware supports out-loop scaler output in YUV420SP format.
  0: Does not support.
  1: Supports.
- u32 vsSupport

  Whether the hardware supports video stabilization.
  0: Does not support.
  1: Supports.
- u32 OSDSupport

  Whether the hardware supports OSD.
  0: Does not support.
  1: Supports.
- u32 maxOsdNum

  The maximum number of OSD rectangles supported by the hardware.
- u32 MosaicSupport

  Whether the hardware supports mosaic.
  0: Does not support.
  1: Supports.
- u32 maxMosaicNum

  The number of mosaic rectangles supported by the hardware.
- u32 mosaicVersion

The version of of mosaic.
0: 64x64 for HEVC, 16x16 for H264.
1: 8x8/16x16/32x32/64x64/128x128/ for H264 and HEVC.

- u32 osdBlendVersion

  Hardware OSD alpha blending Version in the pre-processing pipeline.
  0: Does not support.
  1: Supports.

- u32 OSDMapSupport

  Whether the hardware supports picture mosaic control by map buffer in DDR or not. The mosaic information of every 8x8 blocks are specified in the map buffer.
  0: Does not support.
  1: Supports.

- u32 OSDMapVersion

  The OSD Map version.
  0: 4bits for each 8x8.
  1: 4 bits for each 4x4.

- u32 tile8x8FormatSupport

  Whether the hardware supports 8x8 tiled input formats.
  For more information about input formats, refer to Hantro VC9000E v1x Hardware Features.

- u32 tile4x4FormatSupport

  Whether the hardware supports 4x4 tiled input formats.
  For more information about input formats, refer to Hantro VC9000E v1x Hardware Features.

- u32 customTileFormatSupport

  Whether the hardware supports tiled input formats.
  For more information about input formats, refer to Hantro VC9000E v1x Hardware Features.

- u32 rgb24bitFormatSupport

  Whether the hardware supports RGB888 or BGR888 input formats.
  0: Does not support.
  1: Supports.

- u32 prpSbiSupport

  Whether the hardware supports FLEXA SBI in the pre-processing pipeline.
  0: Does not support.
  1: Supports.

- u32 ufbcSupport

  100: AFBC 200: PVRIC 400: DEC400

- u32 superTileXSupport

  Whether the pre-processing module supports the X-major super tile input format that is output by VeriSilicon GPU IPs.
  0: Does not support.
  1: Supports.

- u32 tile64x4_8bFormatSupport

Whether the pre-processing module supports input formats YUV420SP8b_Tile64x4_A16N and YVU420SP8b_Tile64x4_A16N.
0: Does not support.
1: Supports.

- u32 tile32x4_10bFormatSupport

  Whether the pre-processing module supports input format YUV420SP10bWH_Tile32x4_A16N.

- u32 prpAnalyzerSupport

  Whether the hardware supports input analyzer in the pre-processing pipeline.
  0: Does not support.
  1: Supports.

- u32 prpMirrorSupport

  Whether the hardware supports mirror in pre-processing pipeline.
  0: Does not support.
  1: Supports.

- u32 VideoPX

  HEVC/H264 performance for VC8000LE or VC9000LE 1: 4k15fps @ 520Mhz 4: 4k60fps @ 520Mhz.

- u32 meExternSramSupport

  Whether the hardware supports external SRAM.
  0: Does not support.
  1: Supports.

- u32 busWidth

  The master data bus width.
  0: 32 bits 1: 64bits 2: 128 bits 3: 256bit.

- u32 busType

  Not used any more.

- u32 maxAXIAlignment

  Reserved.

- u32 axi_burst_align_wr_common

  AXI alignment setting for writing common data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_wr_stream

  AXI alignment setting for writing bit streams.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_wr_chroma_ref

  AXI alignment setting for writing chroma reconstructed frame data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_wr_luma_ref

  AXI alignment setting for writing luma reconstructed frame data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_rd_common

  AXI alignment setting for reading common data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_rd_prp

  AXI alignment setting for reading pre-processing data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_rd_ch_ref_prefetch

  AXI alignment setting for reading chroma reference frame data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_rd_lu_ref_prefetch

  AXI alignment setting for reading luma reference frame data.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_wr_cuinfo

  AXI alignment setting for writing CU information.
  6: AXI aligned to 64.
  8: AXI aligned to 256.

- u32 axi_burst_align_wr_4n_recon

  AXI alignment setting for reconstructed frames.
  align to 4n.

- u32 axi_burst_align_rd_4n_ref

  AXI alignment setting for reference frames.
  align to 4n.

- u32 SaoBufOptSupport

  Optimize the extra line buffer of SAOE by change the data size between DBF and SAOE.

- u32 lowLatClkGate

  Whether to have SW registers to control the clock gating behavior when low latency hanshake stall the encoding pipeline.

- u32 h264NalRefIdc2bit

  The number of bits used to write the nal_ref_idc syntax element of H.264.
  0: 1 bit
  1: 2 bits.

- u32 av1ExtensionFlag

  Whether the hardware supports the av1_extension_flag register which controls the syntax in OBU_Header.
  0: Not supported.
  1: Supported.

- u32 av1CarryOpt

  Where hardware do carry propogate in EMC or extra module. 0: Use extra module to do carry propogate calculation.
  1: Do carry propogate calculation in EMC module.

- u32 roiAbsQpSupport

  Whether the hardware supports ROIs with absolute QP.
  0: Does not support.
  1: Supports.

- u32 ROI8Support

  Whether the hardware supports two or eight ROIs.
  0: Supports two ROIs.
  1: Supports eight ROIs.

- u32 h264CavlcEnable

  Whether the hardware supports context-adaptive variable-length coding (CAVLC).
  0: Does not support.
  1: Supports.

- u32 refRingBufEnable

  Whether the hardware supports linear or ring structure for reference and reconstructed frame buffers for P-frame encoding.
  0: Only supports linear structure.
  1: Supports both linear and ring structure.

- u32 disableRecWtSupport

  Whether the hardware supports disabling reconstructed frame writing.
  0: Does not support.
  1: Supports.

- u32 forceIntraCuSizeOpt

  Whether the hardware uses CU size map from IntraModeSearch block (not decided by PK inter vs inter rdcost) for the Cus which are forced to Intra by GDR/CIR/ROI.
  0: Does not support.
  1: Supports.

- u32 ME1NUseSad

  Whether ME1N use SAD to replace SATD.

- u32 IMSUseSatd4

  Whether IMS use SATD4x4 to replace SATD8x8.

- u32 MEMDPkOrder

  Use new cost PK order in MEMD to improve performance. 0: Cost_QN <=> Min(Skip0,Skip1,Skip2) 1: Min(Cost_QN,Skip0,Skip1,Skip2)

- u32 backgroundDetSupport

  Whether the hardware supports background detection.
  0: Does not support.
  1: Supports. (Only for a few customers)

- u32 P010RefSupport

  Whether the hardware supports using P010 as the reference frame format for 10-bit encoding.
  0: Does not support.
  1: Supports. (Only for a few customers)

- u32 streamBufferChain

Whether the hardware supports stream buffer chain.
0: Does not support.
1: Supports.

- u32 gmvSupport

  Whether the hardware supports global motion vector.
  0: Does not support.
  1: Supports. (Only for a few customers)

- u32 IPCM8Support

  Whether the hardware supports eight IPCM area setting.
  0: Does not support.
  1: Supports.

- u32 av1InterpFilterSwitchable

  Whether the hardware supports AV1 switchable interpolation filter.
  0: Does not support.
  1: Supports. (Only for C-model)

- u32 IframeOnly

  Whether the hardware supports I-frame encoding only.
  0: Does not support.
  1: Supports.

- u32 dynamicRdoSupport

  Whether the hardware supports dynamical choosing of RDO level.
  0: Does not support.
  1: Supports.

- u32 tuneToolsSet2Support

  Whether the hardware supports quality improvement defined in set two.
  0: Does not support.
  1: Supports.

- u32 hevcTemporalMvpSupport

  Whether the hardware supports TMVP for HEVC.
  0: Does not support.
  1: Supports.

- u32 av1TemporalMvpSupport

  Whether the hardware supports TMVP for AV1.
  0: Does not support.
  1: Supports.

- u32 tmvpMcSupport

  Whether the hardware supports TMVP with multiple cores together.
  0: Does not support.
  1: Supports.

- u32 intraReconSupport

  Whether the hardware supports intra mode search with reconstructed frames.
  0: Does not support.
  1: Supports.

- u32 av1IntraReconSupport

  Whether the hardware supports AV1 intra mode search with reconstructed frames.
  0: Does not support.
  1: Supports.

- u32 hevcIntraTrDepth0

  Whether the hardware supports Intra Transform Depth=0 for HEVC.
  0: Does not support.
  1: Supports.

- u32 chDistWeightSupport

  Whether the hardware supports programable chroma distortion weight.
  0: Does not support.
  1: Supports.

- u32 h264RdoCoeffSimpleBinEst

  Whether the hardware supports simplified bin cost estimation in RDO for H264.
  0: Does not support.
  1: Supports.

- u32 realTu32Support

  Whether the hardware supports Real TU32 for HEVC/AV1/VP9.
  0: Does not support.
  1: Supports.

- u32 hevcCuRdoSupport

  Whether the hardware supports the CuTree RDO architecture for HEVC.
  0: Does not support.
  1: Supports.

- u32 av1CuRdoSupport

  Whether the hardware supports the CuTree RDO architecture for AV1.
  0: Does not support.
  1: Supports.

- u32 meqnSimpleBinCost

  Whether the hardware supports simplified cost instead of Meqn binCost.
  0: Does not support.
  1: Supports.

- u32 meqnClipYBits

  Whether the hardware supports Clipped Y direction interpolation.
  0: Does not support.
  1: Supports.

- u32 h264MeqnHevcIntp

  Whether the hardware supports HEVC interpolation method for H264.
  0: Does not support.
  1: Supports.

- u32 streamMultiSegment

  Whether the hardware supports multiple segments for stream buffer.
  0: Does not support.
  1: Supports.

- u32 modSubjPrefer

  Whether the hardware supports the modification of subjective quality improvement for H.264.
  0: Does not support.
  1: Supports.
- u32 twoRefPTMVPSupport

  Whether the hardware supports both the 2RefP and TMVP.
  0: Does not support.
  1: Supports.
- u32 mexnBinCostRefine

  Whether to support bin cost refine algorithm in ME4N/ME2N/ME1N.
- u32 ipdMpmUseTop

  Whether to use top neighbor info for MPM mode of IPD in H264 encoding.

6.5 EWLInitParam_t

The parameters required for EWL initialization.

Data Fields

- u32 clientType

  The encoding format. See CLIENT_TYPE.
- void * context

  The context used for initialization. The context is sent to the application by EWL.
- u32 slice_idx

  The index of the slice node for multi-node VPU.
  This field is valid only for specific versions.
- u32 mmuEnable

  Specifies whether the hardware supports MMU.
  0: Does not support.
  1: Supports.
- char * enc_dev

  The device name of the enc driver.
- char * mem_dev

  The device name of the memalloc driver.
- i32 useVcmd

  Specifies whether to use VCMD, only valid for cmodel.
  0: Not use.
  1: Use.

6.6 EWLLinearMem_t

The information of the memory allocated as a linear buffer.

Data Fields

- u32 * virtualAddress

  The aligned virtual address of the memory for CPU to access.
- ptr_t busAddress

  The aligned bus address of the memory for VPU to access.
- u32 size

  The size of memory, in bytes.
- u32 * allocVirtualAddr

  The original virtual address of the memory for CPU to access.
- ptr_t allocBusAddr

  The original bus address of the memory for VPU to access.
- unsigned long id

  The ID of the buffer.
- u32 mem_type

  The owner and attributes of the memory.
  Bit 23 indicates whether the buffer has sensitive data. See MEM_ATTR.
  Bits [22:16] indicate the usage hints of memory. See EWLMemUsageHint.
  Bits [15:8] indicate which IP is allowed to access the memory.
  Bits [7:0] indicate the attributes of the memory.
- u32 total_size

  The size of the allocated buffer, in bytes.
- void * priv

  Customer-defined field for application use.

6.7 EWLMemoryStatistics_t

Specifies allocated memory information for counting.

Data Fields

- u32 maxHeapMemory

  The maximum memory value.

- u32 memoryValue

  Memory value.

- u32 maxHeapMallocTimes

  The number of times `EWLmalloc()` or `EWLcalloc()` are called.

- u32 averageHeapMallocTimes

  The maximum heapMallocTimes.
  For example, if `EWLmalloc()` is called 10 times and `EWLfree()` is called twice, `averageHeapMallocTimes` and `heapMallocTimes` are eight. If then, `EWLfree()` is called one more time, `heapMallocTimes` is seven, but `averageHeapMallocTimes` is still eight. On the contrary, if `EWLmalloc()` is called one more time, `HeapMallocTimes` becomes nine, larger than the former eight, and `maxHeapMallocTimes` becomes nine as well.

- u32 heapMallocTimes

  The number of times [`EWLmalloc()` - `EWLfree()`] are called.

- u32 maxLinearMemory

  The maximum memory value of a linear buffer.

- u32 linearMemoryValue

  The memory value of linear buffer.

- u32 maxLinearMallocTimes

  The number of linear buffers allocated. The number is counted by how many times `EWLMallocLinear()` or `EWLMallocRefFrm()` are called.

- u32 averageLinearMallocTimes

  The maximum `linearMallocTimes`.
  For example, if `EWLMallocLinear()` is called ten times, and `EWLFreeLinear()` is called twice, `averageLinearMallocTimes` and `linearMallocTimes` are eight. If then, `EWLFreeLinear()` is called one more time, `linearMallocTimes` is seven, but `maxLinearMallocTimes` is still eight. On the contrary, if `EWLMallocLinear()` is called one more time, `linearMallocTimes` becomes nine, larger than the former eight, and `maxLinearMallocTimes` becomes nine as well.

- u32 linearMallocTimes

  The number of linear buffers that are not released when an instance is released. This number is counted by subtracting the number of times `EWLMallocLinear()` is called from the number of times `EWLFreeLinear()` is called.

6.8 EWLResource_t

Information about reserved resources.

Data Fields

- EWLResourceVcmdBuf vcmdbuf

  The information about the VCMD buffer, when reserving buffer resources.

6.9 EWLResourceVcmdBuf

The information about a VCMD buffer, when reserving buffer resources.

Data Fields

- u16 size

  [in] The buffer size to reserve.
- u16 id

  [out] The ID of the VCMD buffer.
- ptr_t status_ba

  [out] The bus address of the status buffer.
- u32 * cmdbuf_va

  [out] The virtual address of the VCMD buffer.
- u32 core_mask

  [in] The mask that indicates which core is selected.
- u32 priority

  [in] The priority of the selected VCMD buffer.

6.10 EWLWaitJobCfg_t

The job configuration.

Data Fields

- u32 waitCoreJobid

  The job ID maintained by the kernel driver.
- u32 dec400_enable

  The status of DEC400.
  1: Bypassed.
  2: Enabled.
- void * dec400_data

  The DEC400.
- CoreWaitCallBackFunc dec400_callback

  dec400 callback resvered for furture
- u32 axife_enable

  The status of AXI_FE.
  0: Disabled.
  1: Enabled.
  2: Bypassed.
  3: Security mode.
- CoreWaitCallBackFunc axife_callback

  The AXI_FE callback function.
- u32 l2cache_enable

  The status of L2Cache.
  0: Disabled.
  1: Enabled.
- void * l2cache_data

  The L2Cache.
- CoreWaitCallBackFunc l2cache_callback

  The L2Cache callback function.
- u32 ufbcMode

  The core mode of UFBC.
  0: UFBC disable.
  1: Use the core of AFBC version 0.(only support 32x8 superblock).
  2: Use the core of AFBC version 1.(Support 32x8 and 16x16 superblock).
  3: Use the core of Dec400.
  4: Use the core of PVRIC.
- CoreWaitCallBackFunc2 ufbc_callback

  The UFBC callback. See `EncUfbcAsicStop`.

6.11 sync_mem

The address information of the buffer for test verification of memory synchronization.
NOTE: This structure can be overwritten.

Data Fields

- u32 * dev_va

  The aligned virtual address of the buffer from the host view.

- ptr_t dev_pa_alloc

  The aligned physical address of the buffer from the host view.

- u32 * dev_va_alloc

  The original virtual address of the buffer in device from the host view.

Appendix: Glossary

The following table describes the terms used in this document. For additional HEVC acronyms, see http://www.hevcbook.de/acronyms/.

Term	Description
1080p	high-definition resolution of 1920 x 1080 progressive video
720p	high-definition resolution of 1280 x 720 progressive video
ABR	average bit rate
AIoT	artificial intelligence of things
APB	Advanced Peripheral Bus
API	application programming interface
ARIB	Association of Radio Industries and Businesses
ASIC	application-specific integrated circuit
AV1	AOMedia Video 1
AVC	Advanced Video Coding (H.264)
AXI	Advanced eXtensible Interface
bps	bits per second
CABAC	context-adaptive binary arithmetic coding
CAVLC	content-adaptive variable-length coding
CB	coding block
CBR	constant bit rate
CIF	Common Interchange Format (352 x 288 pixels)
CIR	cyclic intra refresh
CPB	coded picture buffer
CPU	central processing unit
CQP	constant quantization parameter
CRF	constant rate factor
CSC	color space conversion
CTB	coding tree block
CTU	coding tree unit

Term	Description
CU	coding unit
CVBR	constrained variable bit rate
DDR	double data rate
DRM	digital rights management
EFBC	external frame buffer compression
EOS	end of sequence
EWL	encoder wrapper layer
FPS	frames per second
GDR	gradual decoder refresh
GOP	group of pictures
HD	high definition
HDTV	high-definition television
HEIF	High Efficiency Image File Format
HDR	high dynamic range
HEVC	High Efficiency Video Coding, a video coding standard (H.265) developed by JCT-VC
HRD	hypothetical reference decoder
IDR	Instantaneous Decoder Refresh
IPCM	intra pulse code modulation
ITU	International Telecommunication Union
ITU-T	ITU Telecommunication Standardization Sector
ITU-R	ITU Radiocommunication Sector
Kbps	kilobits per second
LCU	largest CU
LTR	long-term reference
MAD	mean absolute difference
MB	macroblock
MMU	memory management unit
MPEG-4	Moving Picture Experts Group 4
MV	motion vector
NAL	network abstraction layer
NALU	NAL unit, the smallest decodable unit of a stream
OBU	open bitstream unit
OSD	on-screen display
PAL	Phase Alternate Line (720 x 576 pixels resolution)
POC	picture order count
PPS	picture parameter set
PSNR	peak signal to noise ratio

Term	Description
PU	prediction unit
QCIF	Quarter CIF (176 x 144 pixels)
QVGA	Quarter Video Graphics Array (320 x 240 pixels)
QP	quantization parameter
RC	rate control
RDO	rate-distortion optimization
RDOQ	rate-distortion optimized quantization
REE	rich execution environment
RFC	reference frame compression
RGB	red, green, blue color space representation
ROI	region of interest
RPS	reference picture set
SAO	sample adaptive offset
SBI	segment buffer interface
SEI	supplemental enhancement information
SMPTE	Society of Motion Picture and Television Engineers
SPS	sequence parameter set
SSIM	structural similarity index measure
STR	short-term reference
sub-QCIF	Sub Quarter CIF (128 x 96 pixels resolution)
SVC-T	temporal scalable video coding
SXGA	Super Extended Graphics Array (1280 x 1024 pixels resolution)
TU	transformation unit
TV	television
VBR	variable bit rate
VBV	video buffer verifier
VGA	Video Graphics Array (640 x 480 pixels resolution)
VLC	variable-length code
VPS	video parameter set
VUI	video usability information
YCbCr	color space with one luma and two chroma components, used for digital encoding
YUV	color space with one luma and two chroma component, used for either digital or analog encoding

Document Revision History

This section describes top level differences in the versions of this document.

NOTE: This document is not necessarily updated for each patch or minor revision. The information in this document tends to be stable across a revision (nnn) series.

Document Revision	Date	Compatible Core	Description
1.20	2024-03-14	VC9000E	- Updated the distribution level from A to B. - Updated the document name and template. - Added Chapters *Linux Kernel Mode Drivers* and *Performance Checking and Issue Debugging*. - Added macros `ASIC_STATUS_POLL_SLICEINFO_TIMEOUT` and `ASIC_STATUS_UFBC_DEC_ERR` in Section *Core Information*. - Added enumerations `EWLMemUsageHint` and `EWLMemAttribute` in Section *Memory Management*. - Removed the parameter `pixel_format` from EWLGetDec400Attribute(). - Replaced enumerator `EWL_CLIENT_TYPE_EFBC` with `EWL_CLIENT_TYPE_UFBC` for `CLIENT_TYPE`. - Added EWLGetClientVcmdStatusBufOffset(). - Removed EWLCopyDataToCmdbuf().

Document Revision	Date	Compatible Core	Description
			- Added parameters `interrupt_ctrl` and `client_type` to EWLSetReserveBaseData(). - Added parameter `inst` to EWLGetVCMDSupport(). - Addded members `dec400_data`, `ufbcEnable`, and `ufbc_callback` to structure EWLCoreWaitJob_t. - Added members `deNoiseEnabled`, `mosaicVersion`, `ufbcSupport`, `superTileXSupport`, `tile64x4_8bFormatSupport`, `tile32x4_10bFormatSupport`, `prpAnalyzerSupport`, `av1TemporalMvpSupport`, `tmvpMcSupport`, `av1IntraReconSupport`, `av1CuRdoSupport`, and `modSubjPrefer` to EWLHwConfig_t. - Replaced member `TemporalMvpSupport` with `hevcTemporalMvpSupport` in EWLHwConfig_t. - Added members `mmuEnable`, `enc_dev`, `mem_dev`, and `useVcmd` to EWLInitParam_t. - Added members `cmdbuf_va`, `core_mask`, `priority` to EWLResourceVcmdBuf. - Added members `dec400_callback`, `ufbcEnable`, and `ufbc_callback` to EWLWaitJobCfg_t. - Updated the data type of `ctx` to `const void` in EWLReadAsicID(), EWLFetCoreNum(), and EWLReadAsicConfig().

Document Revision	Date	Compatible Core	Description
1.10	2023-06-30	VC9000E	- Reconstructed the document and refined descriptions. - Removed the following sections: *Supported Standards*, *Compatible Hardware*, *Deprecated Functions*, and *Memory*. - Added Section *Memory Allocation in Security Mode*, and Section *EWLMemoryStatistics_t*. - Regrouped the functions into the categories of *Core Information*, *Memory Management*, and *EWL API*. - Added return values for `EWLSyncMemData()` and `EWLReadRegInit()`. - Added macros and their descriptions. - Added the following functions: `EWLGetCoreNum()`, `EWLCheckCutreeValid()`, `EWLGetDec400CustomerID()`, `EWLGetDec400Attribute()`, `EWLGetAXIFeCfg()`, `EWLFreeRefFrm()`, `EWLFreeLinear()`, `EWLcalloc()`, `EWLfree()`, `EWLmemcpy()`, `EWLmemset()`, `EWLmemcmp()`, `EWLGetDec400Coreid()`, `EWLGetVcmdVersionId()`, `MapAsicRegisters()`, `EWLGetClientType()`, `EWLGetCoreTypeByClientType()`, `EWLGetPerformance()`, `EWLWriteRegbyClientType()`, `EWLWriteBackRegbyClientType()`, `EWLReadRegbyClientType()`, `EWLDisableHW()`, `EWLReadRegInit()`, `EWLReserveCmdbuf()`, `EWLLinkRunCmdbuf()`,

Document Revision	Date	Compatible Core	Description
			EWLWaitCmdbuf(), EWLGetRegsByCmdbuf(), EWLReleaseCmdbuf(), EWLSetReserveBaseData(), and EWLGetVCMDSupport(). - Added MEM_ATTR. - Added EWL_CLIENT_TYPE_EFBC to CLIENT_TYPE. - Added out_poll_sliceinfo_status to *EWLCoreWaitJob_t*. - Added JpegDHTGenerate, JpegExternalDHT, H264YUV422Support, HevcYUV444Support, and H264YUV444Support to, and removed encStreamSegmentSupport from EWLHwConfig_t. - Renamed DDRLowlatencySupport as prpLowLatencySignalbyDDR, and linBufferEnable as prpLowLatencySupport.
1.00	2017-03-28	VC9000E	Initial release.